



EMERGENCY VEHICLE OBJECT DETECTION FOR TRAFFIC LIGHT OPTIMIZATION



A PROJECT REPORT

Submitted by

P. SESEDHAR REDDY	(622021104076)
P. VAMSI KRISHNA	(622021104077)
G. POOVARASAN	(622021104079)

In partial fulfilment for the award of the degree
of
BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

PAAVAI COLLEGE OF ENGINEERING
PACHAL, NAMAKKAL - 637018

ANNA UNIVERSITY: CHENNAI 600 025

MAY- 2025

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**EMERGENCY VEHICLE OBJECT DETECTION FOR TRAFFIC LIGHT OPTIMIZATION**” is the Bonafide work of “**P.SESEDHARREDDY (622021104076), P.VAMSIKRISHNA (622021104077), G.POOVARASAN (622021104079)**” who carried out the project work under my supervision.

SIGNATURE

Mr. R. MURUGESAN, M.E, (Ph.D.),
HEAD OF THE DEPARTMENT,
PROFESSOR,
Department of Computer
Science and Engineering
Paavai College of Engineering.

SIGNATURE

Mrs. B.M. BRINDA, M.E, (Ph.D.),
SUPERVISOR, ASSOCIATE
ASSISTANT PROFESSOR,
Department of Computer
Science and Engineering
Paavai College of Engineering.

Certified that candidate was examined in the final project viva-voice examination held on_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our profound gratitude with pleasure to our most respected **Chairman Shri. CA. N.V. Natarajan, B. Com, FCA.**, and to our beloved **Correspondent Smt. N. Mangai Natrajan, M.Sc.**, for giving motivation and providing all necessary facility for the successful completion of this project.

It is our privilege to thank our beloved Director - Admin **Dr. K. K. Ramasamy, M.E, Ph.D.**, for their moral support and deeds in bringing out this project successfully.

We extend our sincere gratitude to thank our Principal **Dr. V. Hariharan, M.E, Ph.D.**, for their moral support and deeds in bringing out this project successfully.

We extend our gratefulness to **Mr. R. Murugesan, M.E, (Ph.D.)**, Associate Professor, Head of the Department. Department of Computer Science and Engineering, for his encouragement and constant support in successful completion of project.

We convey our thanks to **Mrs. B.M. Brinda, M.E, (Ph.D.)**, Assistant Professor. Department of Computer Science and Engineering, our supervisor for being more informative and providing suggestions regarding this project all the time.

We would like to express our sincerely thanks and heartiest gratitude to our project coordinator **Mr. K. Srinivasan, M.E.**, Assistant Professor, Department of Computer Science and Engineering, for his meticulous guidance and timely help during project work.

We express thanks to all our department staff members and friends for their encouragement and advice to do the project work with full interest and enthusiasm

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	vii
	PROBLEM STATEMENT	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATION	x
1	INTRODUCTION	1
	1.1 ARTIFICIAL INTELLIGENCE	2
	1.2 VEHICLE OBJECT DETECTION	3
	1.3 METHODS OF OBJECT DETECTION	4
	1.4 NEED FOR THIS SYSTEM	5
	1.5 PROCESS IN VEHICLE DETECTION	6
2	LITERATURE SURVEY	7
3	SYSTEM ANALYSIS	16
	3.1 EXISTING SYSTEM	16
	3.1.1 DISADVANTAGES	17
	3.2 PROPOSED SYSTEM	17
	3.2.1 METHODS TO DETECT VEHICLE OBJECT	18
	3.2.2 ADVANTAGES	19
4	SYSTEM SPECIFICATION	20
	4.1 HARDWARE REQUIRMENTS	20
	4.2 SOFTWARE REQUIRMENTS	20
	4.3 SYSTEM STUDY	21
5	SOFTWARE DESCRIPTION	23

	5.1 FRONTEND PYTHON	23
	5.2 TENSOR FLOW LIBRARIES IN PYTHON	28
	5.3 PYCHARM	29
	5.4 MY SQL	32
6	SYSTEM DESIGN	34
	6.1 SYSTEM ARCHITECTURE	34
	6.2 DATA FLOW DIAGRAM	35
	6.3 UML DIOAGRAMS	37
	6.3.1 USE CASE DIAGRAM	37
	6.3.2 CLASS DIAGRAM	38
	6.3.3 SEQUENCE DIAGRAM	39
	6.3.4 ACTIVITY DIAGRAM	40
7	PROJECT DESCRIPTION	41
	7.1 MODULES DESCRIPTION	41
	7.1.1 DATASETS COLLECTION	41
	7.1.2 VEHICLE IMAGE	42
	7.1.3 AUDIO DATASETS	43
	7.1.4 MODEL CONSTRUCTION	44
	7.1.5 REAL TIME INTERFACE	46
	7.1.6 CLASSIFY THE VEHICLES	46
8	SYSTEM TESTING	48
	8.1 TESTING PROCESS	48
	8.2 TESTING OBJECTIVES	49
	8.2.1 UNIT TESTING	49
	8.2.2 1 FUNCTIONAL TESTING	50
	8.2.3 ACCEPTANCE TESTING	50
	8.2.4 VALIDATION TESTING	50
	8.2.5 INTEGRATION TESTING	51

9	CONCLUSION AND FUTURE ENHANCEMENT	52
	9.1 CONCLUSION	52
	9.2 FUTURE ENHANCEMENTS	53
10	APPENDIX	54
	10.1 SOURCE CODE	54
	10.2 SCREENSHOTS	75
11	REFERENCES	82

ABSTRACT

In modern urban environments, efficient traffic management plays a crucial role in ensuring smooth vehicular movement and minimizing congestion. Traditional traffic light systems operate on fixed timers, which often fail to adapt to real-time traffic conditions, leading to unnecessary delays, particularly for emergency vehicles like ambulances. This project introduces an intelligent Traffic Light Optimization System that leverages deep learning models to detect emergency vehicles using both image and audio inputs. By integrating YOLO (You Only Look Once) for vehicle detection and a CNN (Convolutional Neural Network) for audio classification, the system can accurately recognize ambulances in real-time based on visual and auditory cues. The system processes data captured by cameras and microphones, classifies the information, and optimizes traffic signals accordingly, ensuring faster passage for emergency vehicles while maintaining traffic flow efficiency.

This system utilizes datasets sourced from Roboflow for vehicle images and Kaggle for audio data, ensuring robust model training. The processed vector data is classified, and upon detecting an ambulance, the system communicates with a Traffic Light Controller to prioritize signal changes. This approach significantly enhances emergency response times, reduces congestion, and improves road safety.

PROBLEM STATEMENT

In urban and high-traffic areas, the timely and efficient movement of emergency vehicles such as ambulances, fire trucks, and police cars is critical. Delays caused by traffic congestion or the inability of other drivers and traffic control systems to recognize and react to approaching emergency vehicles can result in severe consequences, including loss of life and property.

Current traffic monitoring and control systems lack real-time, automated mechanisms to detect and respond to the presence of emergency vehicles on the road. This leads to ineffective traffic management, delayed emergency response times, and increased risk of accidents.

Emergency services such as ambulances, fire trucks, and police vehicles play a critical role in responding to life-threatening situations. However, their rapid response capabilities are often hindered by urban traffic congestion, uncooperative drivers, and the lack of intelligent traffic systems that can dynamically adapt to their presence. Despite having visual and audio cues (e.g., sirens and flashing lights), human error or distraction often prevents other drivers and automated systems from recognizing these vehicles in time. A system that can detect emergency vehicles in real-time using visual input from cameras (e.g., traffic surveillance, in-vehicle dashcams, drones) can drastically improve response times and public safety.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
6.1	ARCHITECTURE DIAGRAM	34
6.2	DATA FLOW DIAGRAM LEVEL 0	36
6.3	DATA FLOW DIAGRAM LEVEL 1	36
6.4	DATA FLOW DIAGRAM LEVEL 2	37
6.5	USE CASE DIAGRAM	37
6.6	CLASS DIAGRAM	38
6.7	SEQUENCE DIAGRAM	39
6.8	ACTIVITY DIAGRAM	40
7.1	YOLO	44
7.2	CNN ARCHITECTURE	46
10.1	CONFUSION MATRIX	75
10.2	NORMALIZED CONFUSION MATRIX	75
10.3	CURVE GRAPH	76
10.4	LABEL GRAPH	76
10.5	CORRELOGRAM GRAPH	77
10.6	PRECISION CURVE GRAPH	77
10.7	PRECISION-RECALL GRAPH	78
10.8	RECALL CONFERENCE CURVE	78
10.9	RESULTS GRAPH	79
10.10	IMAGE DETECTION	79
10.11	SOUND RECOGNITION	80
10.12	UPLOADING SOUND RECOGNITION	80
10.13	EMERGENCY VEHICLE	81

	DETECTION	
10.14	EMERGENCY VEHICLE NOT DETECTED	81

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	- ARTIFICIAL INTELLIGENCE
API	- APPLICATION PROGRAMMING INTERFACE
CNN	- CONVOLUTIONAL NEURAL NETWORK
GPS	- GLOBAL POSITIONING SYSTEM
GPU	- GRAPHICS PROCESSING UNIT
GUI	- GRPHICAL USER INTERFACE
IDE	- INTEGRATED DEVELOPMENT
IOT	- INTERNET OF THINGS
ITS	- INTELLIGENT TRANSPORTATION SYSTEM
RFID	- RADIO FREQUENCY IDENTIFICATION
RNN	- RECURRENT NEURAL NETWORK
UML	- UNIFIED MODELLING LANGUAGE
V2I	- VEHICLE TO INFRASTRUCTURE
V2X	- VEHICLE TO EVERYTHING
YOLO	- YOU ONLY LOOK ONCE

CHAPTER 1

INTRODUCTION

With the increasing number of vehicles on roads, urban traffic congestion has become a major challenge, affecting both daily commuters and emergency response teams. Traffic lights in most cities operate on predefined timing sequences, which fail to account for dynamic traffic conditions. This inefficiency often results in significant delays, particularly for emergency vehicles such as ambulances, which require uninterrupted passage to hospitals. In many cases, ambulances are stuck in traffic, causing life-threatening delays for critical patients. The inability of conventional traffic management systems to detect and respond to ambulances in real time highlights the urgent need for an intelligent traffic control mechanism. The current traffic management infrastructure lacks real-time adaptability to emergency vehicles, which is a major drawback in ensuring rapid medical response. Manual interventions by traffic police are often required to clear routes, but this process is not always feasible, especially in highly congested areas.

Existing smart traffic systems primarily rely on RFID-based or GPS-based tracking, which, while useful, are not universally implemented across all ambulances. Moreover, audio-based siren detection techniques are rarely integrated into these systems, making it difficult to identify emergency vehicles in real-time. These limitations demand a robust, automated system capable of detecting ambulances efficiently through multiple input modalities. To address these challenges, this project proposes an AI-driven Traffic Light Optimization System that employs computer vision and deep learning techniques to detect emergency vehicles using both image and audio inputs. The system utilizes YOLO (You Only Look Once) for vehicle detection and a CNN-based model for siren sound recognition, ensuring accurate and real-

time ambulance identification. Upon detection, the system automatically adjusts traffic signals, allowing emergency vehicles to pass without unnecessary delays. By integrating modern AI techniques with real-time traffic management, this project aims to reduce emergency response times, enhance road safety, and improve overall traffic flow efficiency in urban areas.

1.1 ARTIFICIAL INTELLIGENCE

Artificial Intelligence (AI) refers to the development of computer systems that can perform tasks typically requiring human intelligence. These tasks include decision-making, problem-solving, language understanding, vision, and learning. AI systems are designed to mimic cognitive functions such as reasoning, learning from experiences, and adapting to new inputs. AI's goal is to create machines that can function autonomously and efficiently in complex environments, often surpassing human capabilities in specific tasks. The concept of AI is rooted in the idea of building machines that can simulate intelligent behavior. The field has evolved significantly over the years, from rule-based expert systems to the sophisticated machine learning algorithms used today. Machine learning, a subset of AI, involves training systems to recognize patterns and make predictions based on data, without being explicitly programmed for each specific task. Deep learning, a further advancement, uses multi-layered neural networks to handle large amounts of unstructured data, achieving human-like performance in areas such as speech and image recognition. AI can be broadly categorized into narrow AI and general AI.

Narrow AI refers to systems that are designed to perform specific tasks, such as facial recognition or language translation. These systems excel at their designated functions but lack the broader cognitive abilities that

humans possess. On the other hand, general AI aims to build machines capable of performing any intellectual task that a human being can do. While general AI is still a distant goal, advancements in narrow AI have led to groundbreaking applications in industries like healthcare, finance, and autonomous driving. The impact of AI is profound and transformative, as it has the potential to revolutionize many aspects of life and work. It offers improvements in automation, decision-making, and problem-solving, leading to efficiencies in various sectors. However, AI also presents challenges, particularly regarding ethical considerations, privacy concerns, and job displacement. As AI continues to develop, addressing these challenges will be crucial to ensuring its responsible and beneficial integration into society.

1.2 VEHICLE OBJECT DETECTION

The vehicle object detection component of the system is designed to identify ambulances from live camera feeds using advanced computer vision techniques. This is achieved by leveraging the YOLO (You Only Look Once) deep learning model, which is renowned for its speed and accuracy in real-time object detection tasks. YOLO processes entire images in a single forward pass, dividing the image into a grid and predicting bounding boxes and class probabilities directly. This makes it highly efficient for detecting emergency vehicles like ambulances amidst dense traffic, enabling the system to make quick and informed decisions. YOLO is trained on a curated dataset of various vehicles, with a particular focus on ambulances, using labeled images sourced from platforms like Roboflow. During operation, the system captures frames from surveillance cameras and passes them through the YOLO model, which returns the location and identity of detected objects. If an ambulance is identified with high confidence, the detection is forwarded to the traffic signal controller. This approach ensures that ambulances are detected accurately and

promptly, even when surrounded by other vehicles, thus significantly improving emergency response efficiency.

1.3 METHODS OF OBJECT DETECTION

- **YOLO (You Only Look Once):** The proposed system utilizes the YOLO algorithm for real-time vehicle detection due to its high speed and accuracy. YOLO treats object detection as a regression problem and predicts bounding boxes and class probabilities directly from the full image in a single neural network pass. This enables fast processing of live video streams, making it suitable for detecting ambulances in dynamic traffic environments.
- **Image Preprocessing:** Before feeding frames into the YOLO model, input images from traffic cameras are resized and normalized to meet the model's input requirements. Data augmentation techniques such as flipping, rotation, and brightness adjustment are used during training to improve model robustness across various lighting and weather conditions.
- **Object Classification and Localization:** YOLO divides the input image into a grid and assigns each cell the task of predicting bounding boxes and class scores. If an ambulance is present, YOLO outputs the coordinates of the bounding box and assigns it a high confidence score, allowing the system to accurately distinguish ambulances from other vehicles in the scene.
- **Post-Processing (Non-Maximum Suppression):** After prediction, post-processing techniques like **Non-Maximum Suppression (NMS)** are applied to eliminate redundant bounding boxes with lower

confidence scores. This ensures that only the most accurate bounding box for the ambulance is retained, minimizing false detections.

- **Integration with Traffic Control:** Once an ambulance is successfully detected using YOLO, the output is used to trigger a signal change in the traffic light controller. This real-time feedback mechanism ensures that the detected ambulance receives a green signal, allowing it to move through the intersection with minimal delay.

1.4 NEED FOR THIS SYSTEM

In modern urban environments, traffic congestion has become a critical issue, especially when it hinders the movement of emergency vehicles like ambulances. Traditional traffic signal systems operate on fixed timers and are not equipped to recognize or respond to emergency scenarios in real time. As a result, ambulances are often delayed while navigating through traffic, which can lead to life-threatening consequences for patients requiring urgent medical attention. There is a growing necessity for smart systems that can dynamically adapt to such high-priority situations. This project addresses the urgent need for an intelligent traffic light optimization system that detects ambulances using both visual and audio data. By leveraging deep learning models such as YOLO for image-based vehicle detection and CNN for siren-based audio classification, the system can identify emergency vehicles in real time and adjust traffic signals accordingly. This dual-modality approach ensures greater reliability even in low-visibility conditions, ultimately reducing emergency response times and saving lives. The integration of such a system into urban infrastructure is essential to improve road safety, efficiency, and overall traffic management.

1.5 PROCESS IN VEHICLE OBJECT DETECTION

The process of vehicle object detection in the proposed system involves several key steps to ensure real-time and accurate identification of emergency vehicles, particularly ambulances. Here's a breakdown of the process:

- **Data Acquisition:** The system uses live camera feeds installed at traffic junctions to capture continuous video frames of oncoming traffic. These frames serve as the input for the object detection process.
- **Preprocessing:** The captured images are resized and normalized to fit the input requirements of the YOLO (You Only Look Once) model. This step enhances detection speed and accuracy by eliminating noise and standardizing input dimensions.
- **Object Detection with YOLO:** The YOLO deep learning model is applied to the pre-processed image data. It divides the image into a grid and predicts bounding boxes and class probabilities for each section. This allows the system to detect and identify different types of vehicles, with a specific focus on recognizing ambulances based on shape, size, and label.
- **Bounding Box and Classification:** Once vehicles are detected, bounding boxes are drawn around them, and classification scores are assigned. If an ambulance is identified with high confidence, the system triggers the next phase of action.
- **Signal Communication:** Upon ambulance detection, the system sends a command to the traffic light controller to prioritize the signal direction for that vehicle, allowing it to pass through the junction quickly and safely.

CHAPTER 2

LITERATURE SURVEY

2.1 TITLE: IoT-Based Emergency Vehicle Services in Intelligent Transportation System [4]

DESCRIPTION: This paper explores an Internet of Things (IoT)-based solution for optimizing emergency vehicle services within an Intelligent Transportation System (ITS). The authors propose a framework that facilitates real-time tracking, communication, and coordination of emergency vehicles to reduce response time and improve overall efficiency. The system leverages sensors, GPS, and wireless communication technologies to gather and transmit data between emergency services and traffic management systems. The architecture includes modules for vehicle identification, route optimization, traffic light control, and priority-based lane management. A cloud-based platform is integrated to process data and make informed decisions quickly.

2.1.1 ADVANTAGES:

- Real-time tracking and response
- Scalable and flexible integration with smart city infrastructure
- Significant improvement in response times for emergency vehicles

2.1.2 DISADVANTAGES:

- Dependent on uninterrupted network connectivity
- Potential security risks in handling sensitive emergency data
- Requires substantial initial infrastructure investment

2.2 TITLE: V2X-Based Highly Reliable Warning System for Emergency Vehicles [2]

DESCRIPTION: This paper discusses the development of a Vehicle-to-Everything (V2X)-based warning system to enhance the safety and efficiency of emergency vehicles. The authors introduce a highly reliable communication protocol that enables emergency vehicles to send real-time warnings to surrounding vehicles, traffic management systems, and pedestrians, thereby reducing the risk of accidents. The system integrates V2X communication with advanced warning algorithms to ensure that emergency vehicles can navigate through traffic more safely. The paper emphasizes the use of a highly reliable system capable of functioning under various challenging conditions, including network congestion and unreliable communication channels. Furthermore, the authors present a detailed analysis of system performance under different scenarios using simulations and real-world data.

2.2.1 ADVANTAGES:

- Enhanced safety through real-time communication with surrounding vehicles
- High reliability even under challenging communication conditions
- Scalable system compatible with existing infrastructure

2.2.2 DISADVANTAGES:

- Reliance on the availability of V2X communication networks
- Requires continuous data transmission, which can be bandwidth-intensive
- Performance may degrade in highly congested network environments.

2.3 TITLE: Acoustic Based Emergency Vehicle Detection Using Ensemble of Deep Learning Models [13]

DESCRIPTION: This paper presents an innovative approach to detecting emergency vehicles using acoustic signals. The authors propose the use of an ensemble of deep learning models to identify siren sounds in noisy urban environments. The system combines multiple deep learning architectures to improve detection accuracy by overcoming the challenges posed by overlapping sound sources and ambient noise. The acoustic features extracted from the audio signals are fed into several neural networks, including Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), which work together to classify sirens. The ensemble model outperforms single models by leveraging the strengths of different network types. The paper also discusses how this system can be deployed in real-time traffic management systems to alert other vehicles and pedestrians about the presence of emergency vehicles, improving overall safety and response times.

2.3.1 ADVANTAGES:

- High accuracy in detecting emergency vehicle sirens
- Reduced false positives compared to single deep learning models
- Can be integrated into existing traffic management systems

2.3.2 DISADVANTAGES:

- Requires high-quality microphones and noise-canceling equipment
- May struggle in extremely noisy or dynamic environments
- Computationally intensive due to the use of ensemble models.

2.4 TITLE: Libsignal: An Open Library for Traffic Signal Control [12]

DESCRIPTION: This paper introduces **Libsignal**, an open-source library designed to optimize traffic signal control using machine learning algorithms. The authors aim to improve traffic flow and reduce congestion by dynamically adjusting signal timings based on real-time traffic data. The system leverages various sensors and cameras to collect traffic density information, which is then processed using machine learning models to predict traffic conditions and adjust signal timings accordingly. Libsignal allows for the flexible deployment of different control strategies, such as fixed-time, adaptive, and demand-responsive signal control. The system's modular design makes it adaptable to various traffic management environments and scalable for large urban networks.

2.4.1 ADVANTAGES:

- Reduces congestion by dynamically adjusting traffic signal timings
- Modular and scalable design for flexible deployment
- Open-source library encourages collaboration and further development

2.4.2 DISADVANTAGES:

- Relies on real-time traffic data, which may be challenging to collect accurately in all conditions
- Computationally demanding, requiring significant processing power for real-time adjustments
- Limited by existing infrastructure, requiring updates to sensors and traffic management systems.

2.5 TITLE: Intelligent Road Management System for Autonomous, Non-autonomous, and VIP Vehicles [14]

DESCRIPTION: This paper proposes an intelligent road management system designed to handle autonomous, non-autonomous, and VIP vehicles in a coordinated manner. The system aims to improve road safety and traffic efficiency by prioritizing vehicles based on their type and urgency. Autonomous vehicles are integrated with a vehicle-to-infrastructure (V2I) communication system that allows them to communicate with road sensors and traffic signals, ensuring seamless navigation through smart intersections. Non-autonomous vehicles are incorporated into the system using conventional traffic management technologies, while VIP vehicles are granted priority access through automated traffic signal adjustments. The system is built on a multi-layered architecture that supports real-time data processing and dynamic decision-making.

2.5.1 ADVANTAGES:

- Improves traffic flow by prioritizing different vehicle types based on urgency
- Enhances road safety with real-time dynamic traffic management

2.5.2 DISADVANTAGES:

- High implementation cost due to advanced infrastructure requirements
- Potential cybersecurity vulnerabilities in vehicle communication systems
- Requires continuous updates to maintain compatibility with evolving vehicle technologies.

2.6 TITLE: A Survey of Vehicular Network Systems for Road Traffic Management[10]

DESCRIPTION: This paper surveys various vehicular network systems that aim to enhance road traffic management by leveraging vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) communication technologies. The authors explore the key components and communication protocols of these systems, discussing how they contribute to real-time traffic monitoring, dynamic route planning, and congestion management. By utilizing distributed and centralized network models, vehicular networks allow for effective coordination between vehicles and road infrastructure, enabling optimized traffic flow and safety. The paper also addresses challenges such as network scalability, security, and data privacy, while proposing solutions for efficient communication in urban and rural environments. Real-world case studies and simulations demonstrate the effectiveness of vehicular networks in reducing traffic congestion, preventing accidents, and improving fuel efficiency.

2.6.1 ADVANTAGES:

- Improved traffic flow through real-time vehicle coordination
- Enhanced road safety by preventing accidents and collisions
- Scalability for integration into smart city environments

2.6.2 DISADVANTAGES:

- Vulnerability to security and privacy breaches in vehicular communication
- High infrastructure and maintenance costs for large-scale implementation
- Limited adoption in regions with less developed communication networks

2.7 TITLE: A Comprehensive Survey on Cooperative Intersection Management for Heterogeneous Connected Vehicles [8]

DESCRIPTION: This paper presents an in-depth review of cooperative intersection management systems that aim to optimize traffic flow at intersections for heterogeneous connected vehicles, including autonomous and human-driven cars. The authors explore various strategies, such as cooperative perception, predictive control, and adaptive signal timing, to manage the flow of vehicles through intersections in a more efficient and safe manner. The research highlights the importance of inter-vehicle communication and integration with traffic management systems to enhance cooperative decision-making.

2.7.1 ADVANTAGES:

- Improves safety by coordinating vehicle behavior at intersections
- Reduces delays and congestion at intersections for all vehicle types
- Potential to enhance fuel efficiency and reduce emissions

2.7.2 DISADVANTAGES:

- Requires significant investment in infrastructure upgrades
- Potential interoperability issues between different vehicle types and technologies
- Risk of system failure in the event of communication breakdowns.

2.8 TITLE: Intelligent Transportation Systems Using Roadside Infrastructure: A Literature Survey [6]

DESCRIPTION: This paper provides a comprehensive literature survey on intelligent transportation systems (ITS) that leverage roadside infrastructure to enhance road safety, traffic efficiency, and environmental sustainability. The authors examine various ITS components, including sensors, cameras, and communication networks, that collect and transmit data to manage traffic flow and prevent accidents. The paper focuses on the integration of these systems with traffic management platforms, such as adaptive traffic lights, variable message signs, and incident detection systems, to provide real-time solutions for congestion and safety issues. The authors also discuss the role of artificial intelligence in enabling intelligent decision-making, such as dynamic traffic signal control and predictive traffic modeling.

2.8.1 ADVANTAGES:

- Enhances traffic flow and safety through real-time data processing
- Reduces environmental impact by optimizing traffic signals and reducing idle times
- Potential for seamless integration with future smart city infrastructure

2.8.2 DISADVANTAGES:

- High deployment costs for roadside infrastructure and sensors
- Dependence on continuous network connectivity for real-time updates
- Vulnerability to cyber-attacks and data privacy risks.

2.9 TITLE: Safe Intersection Management with Cooperative Perception for Mixed Traffic of Human-driven and Autonomous Vehicles [1]

DESCRIPTION: This paper focuses on the safe management of intersections involving both human-driven and autonomous vehicles through cooperative perception. The authors propose a system that enables vehicles to communicate and share their perception of the environment, allowing for better decision-making in complex and dynamic traffic scenarios. The system leverages vehicle-to-vehicle (V2V) communication, along with sensors such as cameras and LIDAR, to detect and predict the behavior of other vehicles and pedestrians. The approach enhances situational awareness and enables coordinated actions between vehicles, improving safety and efficiency at intersections. The paper discusses the integration of this cooperative perception system with existing traffic management infrastructure and its potential to address challenges in mixed traffic environments, such as detecting uncooperative human drivers and managing unexpected scenarios.

2.9.1 ADVANTAGES:

- Improved safety at intersections through coordinated vehicle actions
- Increased situational awareness for both autonomous and human-driven vehicles
- Reduces risk of accidents by predicting vehicle and pedestrian behavior

2.9.2 DISADVANTAGES:

- Requires high levels of vehicle cooperation for effectiveness
- Dependent on robust communication systems to function reliably

2.10 TITLE: On the Integration of Enabling Wireless Technologies and Sensor Fusion for Next-Generation Connected and Autonomous Vehicles [3]

DESCRIPTION: This paper explores the integration of enabling wireless technologies and sensor fusion techniques for the development of next-generation connected and autonomous vehicles. The authors focus on the role of 5G networks, Wi-Fi, and Bluetooth in enabling seamless communication between vehicles and infrastructure. Additionally, sensor fusion techniques that combine data from multiple sources, including cameras, radar, LIDAR, and GPS, are discussed as a means to enhance the accuracy and reliability of autonomous driving systems. The paper reviews existing communication protocols and technologies, proposing advancements that can improve real-time data exchange, reduce latency, and support higher levels of vehicle autonomy.

2.10.1 ADVANTAGES:

- Enhanced vehicle autonomy and safety through sensor fusion
- Low-latency communication for real-time decision-making
- Supports seamless interaction between connected vehicles and infrastructure

2.10.2 DISADVANTAGES:

- Requires significant upgrades to existing communication infrastructure
- Potential issues with sensor accuracy in certain environments (e.g., inclement weather)
- High energy consumption for continuous communication and sensor operation

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

In many urban areas today, traffic management systems still rely on fixed-timer traffic lights or simple sensor-based controls that operate without considering the dynamic needs of emergency vehicles. These conventional systems are not capable of identifying or prioritizing ambulances or other emergency services in real time, leading to frequent delays during high-traffic conditions. As a result, ambulances often struggle to move quickly through congested intersections, risking the lives of patients who require urgent medical care. The rigidity of these systems makes them ineffective in handling critical scenarios where time is of the essence.

To address this, some traffic authorities have implemented manual intervention, where traffic police clear the way for ambulances by overriding signal controls or managing traffic flow directly at intersections. While this can be effective in some situations, it is inherently inefficient, slow, and prone to human error. It also requires constant human supervision and cannot scale effectively in cities with high traffic volumes and limited manpower. Manual methods also fail in adverse weather or poor visibility conditions, where human monitoring becomes even more challenging. In recent years, RFID and GPS-based emergency vehicle detection systems have been introduced in certain regions. These systems allow traffic signals to be overridden when an RFID tag or GPS-equipped ambulance is detected nearby. However, these methods come with their own limitations such as high costs, installation complexity, and dependency on specialized hardware both in the vehicles and in the traffic infrastructure. Moreover, these systems do not account for audio-based detection using sirens, which can be critical in low-visibility environments like fog, heavy rain, or night-time. This lack of a

comprehensive and intelligent system for emergency vehicle prioritization creates a significant gap in existing traffic control mechanisms.

3.1.1 DISADVANTAGES

- Fixed traffic light cycles do not adapt to emergency vehicle needs, causing delays.
- Manual traffic management for ambulances is inefficient .
- RFID and GPS-based systems require specialized hardware, increasing costs and complexity.
- Lack of sound recognition prevents the system from identifying ambulances using sirens.

3.2 PROPOSED SYSTEM

The proposed system presents an intelligent and adaptive traffic light optimization framework specifically designed to prioritize emergency vehicles like ambulances. Unlike conventional traffic light systems that rely on fixed time intervals, this approach dynamically adjusts traffic signals in real-time based on the detection of emergency vehicles. To accomplish this, the system integrates YOLO (You Only Look Once) for real-time image- based ambulance detection from live traffic camera feeds. YOLO's fast object detection capability ensures that ambulances are accurately recognized with minimal delay, even in crowded traffic scenarios.

In addition to visual detection, the system incorporates a Convolutional Neural Network (CNN) to analyze audio inputs from roadside microphones and detect the presence of ambulance sirens. This multimodal approach combining both visual and auditory cues enhances the robustness and reliability of the system. It allows for accurate ambulance detection even in situations with low visibility, such as nighttime, fog, or rain, where visual recognition alone may be unreliable. By leveraging deep learning, the system intelligently distinguishes ambulance sirens from other environmental noises.

Upon detecting an ambulance, the system immediately processes the combined data and communicates with the Traffic Signal Controller to adjust the signal phases. It automatically prioritizes the green signal on the ambulance's route and pauses other traffic directions, enabling quick and uninterrupted passage through intersections. This process is fully automated but includes an interactive interface for traffic authorities to monitor and, if necessary, override decisions. The integration of deep learning, real-time data processing, and smart control mechanisms significantly enhances emergency response times, reduces congestion, and promotes safer, more efficient urban mobility.

3.2.1 METHODS TO DETECT VEHICLE OBJECT

The primary method used for vehicle object detection in this project is the YOLO (You Only Look Once) deep learning algorithm. YOLO is a real-time object detection model that identifies multiple objects within a single frame by predicting bounding boxes and class probabilities directly from full images in one evaluation. This makes it highly efficient and suitable for real-time traffic monitoring. The system utilizes YOLO to scan camera feeds and detect vehicles, specifically identifying ambulances by recognizing their unique visual features such as shape, size, and markings.

To enhance detection accuracy in various conditions, the YOLO model is trained on a diverse dataset containing images of vehicles, especially ambulances, sourced from platforms like Roboflow. The model can detect ambulances even in complex environments such as crowded intersections or during adverse weather conditions. YOLO's speed and precision enable it to make quick decisions that are crucial in emergency situations where every second matters.

In addition to visual detection, although not part of the vehicle object detection itself, the system integrates audio classification using CNNs to

further validate the presence of emergency vehicles through siren detection. This multi-modal detection strategy improves reliability, especially when visual data is partially obscured.

3.2.2 ADVANTAGES

- Enhances emergency response times.
- Reduces congestion and improves overall traffic management efficiency.
- Operates in real-time, ensuring quick and adaptive decision-making for emergency situations.
- Provides higher accuracy compared to traditional machine learning-based traffic systems.

CHAPTER 4

SYSTEM SEPCIFICATION

4.1 HARDWARE REQUIREMENTS

- Processor : Intel i5 5 Octa-core
- RAM : 16GB DDR4
- Graphics (GPU) : Intel iRISxe
- Microphones : In-built mic
- Camera : High-resolution cameras
- Speakers : Inbuilt speaker
- Storage : 256 GB SSD

4.2 SOFTWARE REQUIREMENTS

- OS : Windows 11 (64-bit)
- Deep Learning : TensorFlow, Pytorch
- Object Detection : YOLOv11
- Database : MySQL
- Programming Language : Python
- UI Technologies : HTML, CSS, JavaScript

4.3 SYSTEM STUDY

FEASIBILITY STUDY

Feasibility study examines the viability or sustainability of an idea, project, or business. The study examines whether there are enough resources to implement it, and the concept has the potential to generate reasonable profits. In addition, it will demonstrate the benefits received in return for taking the risk of investing in the idea. These studies analyze strengths, weaknesses, opportunities, and threats to determine whether the proposals are cost-effective and beneficial to a company's long-term success. Furthermore, investors can benefit from evaluating the problems and solutions listed in the study and determine whether a proposed project is the right choice for a company.

TECHNICAL FEASIBILITY

This assessment is based on an outline design of system requirements, to determine whether the company has the technical expertise to handle completion of the project. When writing a feasibility report, the following should be taken to consideration:

- A brief description of the business to assess more possible factors which could affect the study
- The part of the business being examined
- The human and economic factor
- The possible solutions to the problem

At this level, the concern is whether the proposal is both technically and legally feasible (assuming moderate cost).[citation needed. The technical feasibility assessment is focused on gaining an understanding of the present technical resources of the organization and their applicability to the expected needs of the proposed system. It is an evaluation of the hardware and software and how it meets the need of the proposed system.

OPERATIONAL FEASIBILITY STUDY

Operational feasibility is the measure of how well a proposed system solves the problems, and takes advantage of the opportunities identified during scope definition and how it satisfies the requirements identified in the requirements analysis phase of system development. The operational feasibility assessment focuses on the degree to which the proposed development project fits in with the existing business environment and objectives with regard to development schedule, delivery date, corporate culture and existing business processes.

FINANCIAL FEASIBILITY

In case of a new project, financial viability can be judged on the following parameters:

- Total estimated cost of the project
- Financing of the project in terms of its capital structure, debt to equity ratio and promoter's share of total cost
- Existing investment by the promoter in any other business
- Projected cash flow and profitability

ORGANIZATION FEASIBILITY

Organization feasibility focuses on the organization's structure, including the legal system, management team's competency, etc. It checks whether the existing conditions will suffice to implement the business idea.

ECONOMIC FEASIBILITY

Economic evaluation is a vital part of investment appraisal, dealing with factors that can be quantified, measured, and compared in monetary terms (Chen 1996). The results of an economic evaluation are considered with other aspects to make the project investment decision as the proper investment appraisal helps to ensure that the right project is undertaken in a manner that gives it the best chances of success.

CHAPTER 5

SOFTWARE DESCRIPTION

5.1 FRONT END: PYTHON

Python is a high-level, interpreted programming language that is widely used in various domains such as web development, scientific computing, data analysis, artificial intelligence, machine learning, and more. It was first released in 1991 by Guido van Rossum and has since become one of the most popular programming languages due to its simplicity, readability, and versatility. One of the key features of Python is its easy-to-learn syntax, which makes it accessible to both novice and experienced programmers. It has a large standard library that provides a wide range of modules for tasks such as file I/O, networking, regular expressions, and more. Python also has a large and active community of developers who contribute to open-source libraries and packages that extend its capabilities.

INTRODUCTION

Python is an interpreted language, which means that it is executed line-by-line by an interpreter rather than compiled into machine code like C or C++. This allows for rapid development and testing, as well as easier debugging and maintenance of code. Python is used for a variety of applications, including web development frameworks such as Django and Flask, scientific computing libraries such as NumPy and Pandas, and machine learning libraries such as TensorFlow and PyTorch. It is also commonly used for scripting and automation tasks due to its ease of use and readability. Overall, Python is a powerful and versatile programming language that is widely used in a variety of domains due to its simplicity, ease of use, and active community.

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in

1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. It provides constructs that enable clear programming on both small and large scales. In July 2018, Van Rossum stepped down as the leader in the language community. Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library. Python interpreters are available for many operating systems.

CPython :

The reference implementation of Python, is open-source software and has a community-based development model, as do nearly all of Python's other implementations. Python and CPython are managed by the non-profit Python Software Foundation. Rather than having all of its functionality built into its core, Python was designed to be highly extensible. This compact modularity has made it particularly popular as a means of adding programmable interfaces to existing applications. Van Rossum's vision of a small core language with a large standard library and easily extensible interpreter stemmed from his frustrations with ABC, which espoused the opposite approach. While offering choice in coding methodology, the Python philosophy rejects exuberant syntax (such as that of Perl) in favor of a simpler, less-cluttered grammar. As Alex Martelli put it: "To describe something as 'clever' is not considered a compliment in the Python culture." Python's philosophy rejects the Perl "there is more than one way to do it" approach to language design in favour of "there should be one—and preferably only one—obvious way to do it".

Python's developers strive to avoid premature optimization, and reject patches to non-critical parts of CPython that would offer marginal increases in speed at the cost of clarity. [When speed is important, a Python programmer

can move time-critical functions to extension modules written in languages such as C, or use PyPy, a just-in-time compiler. CPython is also available, which translates a Python script into C and makes direct C-level API calls into the Python interpreter.

Debugging Python programs is easy: a bug or bad input will never cause a segmentation fault. Instead, when the interpreter discovers an error, it raises an exception. When the program doesn't catch the exception, the interpreter prints a stack trace.

Python also has a large and active community of developers who contribute to a wide range of open-source libraries and tools, making it easy to find and use pre-built code to solve complex problems.

Python has a wide range of applications, including:

Data Science :

Python is one of the most popular languages for data science, thanks to libraries like NumPy, Pandas, and Matplotlib that make it easy to manipulate and visualize data.

Machine Learning:

Python is also widely used in machine learning and artificial intelligence, with libraries like TensorFlow, Keras, and Scikit-learn that provide powerful tools for building and training machine learning models.

Web Development:

Python is commonly used in web development, with frameworks like Django and Flask that make it easy to build web applications and APIs.

Scientific Computing: Python is used extensively in scientific computing, with libraries like SciPy and SymPy that provide powerful tools for numerical analysis and symbolic mathematics.

In addition to its versatility and ease of use, Python is also known for its portability and compatibility. Python code can be run on a wide range of

platforms, including Windows, macOS, and Linux, and it can be integrated with other languages like C and Java.

Overall, Python is a powerful and versatile programming language that is well-suited for a wide range of applications, from data science and machine learning to web development and scientific computing. Its simplicity, readability, and large community of developers make it an ideal choice for beginners and experts alike.

There are two attributes that make development time in Python faster than in other programming languages:

ATTRIBUTE 1 :

Python is an interpreted language, which precludes the need to compile code before executing a program because Python does the compilation in the background. Because Python is a high-level programming language, it abstracts many sophisticated details from the programming code. Python focuses so much on this abstraction that its code can be understood by most novice programmers.

ATTRIBUTE 2 :

Python code tends to be shorter than comparable codes. Although Python offers fast development times, it lags slightly in terms of execution time. Compared to fully compiling languages like C and C++, Python programs execute slower. Of course, with the processing speeds of computers these days, the speed differences are usually only observed in benchmarking tests, not in real-world operations. In most cases, Python is already included in Linux distributions and Mac OS X machines.

One of the strengths of Python is its rich ecosystem of third-party libraries and tools. These libraries provide a wide range of functionality, from scientific computing and data analysis to web development and machine learning. Some popular Python libraries and frameworks include:

NumPy :

A library for numerical computing in Python, providing support for large, multi-dimensional arrays and matrices, along with a large collection of mathematical functions to operate on these arrays.

Pandas:

A library for data manipulation and analysis in Python, providing support for reading and writing data in a variety of formats, as well as powerful tools for manipulating and analyzing data.

Matplotlib:

A plotting library for Python that provides a variety of visualization tools, including line plots, scatter plots, bar plots, and more.

TensorFlow: an open-source machine learning library for Python that provides a variety of tools and algorithms for building and training machine learning models.

Django:

A popular web framework for Python that provides a full-stack framework for building web applications, with support for everything from URL routing to user authentication and database integration.

Python's popularity has also led to a large and active community of developers who contribute to open-source projects and share code and resources online. This community provides a wealth of resources for learning Python, including tutorials, online courses, and forums for asking and answering questions.

Overall, Python is a versatile and powerful programming language that is well-suited for a wide range of applications. Its simplicity, flexibility,

5.2 TENSORFLOW LIBRARIES IN PYTHON

TensorFlow is an open-source machine learning framework developed by Google Brain Team. It is one of the most popular libraries for building and training machine learning models, especially deep neural networks. TensorFlow allows developers to build complex models with ease, including image and speech recognition, natural language processing, and more. One of the key features of TensorFlow is its ability to handle large-scale datasets and complex computations, making it suitable for training deep neural networks. It allows for parallelization of computations across multiple CPUs or GPUs, allowing for faster training times. TensorFlow also provides a high-level API called Keras that simplifies the process of building and training models. TensorFlow offers a wide range of tools and libraries that make it easy to integrate with other Python libraries and frameworks. It has built-in support for data preprocessing and visualization, making it easy to prepare data for training and analyze model performance. One of the major advantages of TensorFlow is its ability to deploy models to a variety of platforms, including mobile devices and the web.

Graph-based computation: TensorFlow uses a graph-based computation model, which allows for efficient execution of computations across multiple devices and CPUs/GPUs.

Automatic differentiation: TensorFlow provides automatic differentiation, which allows for efficient computation of gradients for use in backpropagation algorithms.

High-level APIs: TensorFlow provides high-level APIs, such as Keras, that allow developers to quickly build and train complex models with minimal code.

Preprocessing and data augmentation: TensorFlow provides a range of tools for preprocessing and data augmentation, including image and text

preprocessing, data normalization, and more.

Distributed training: TensorFlow supports distributed training across multiple devices, CPUs, and GPUs, allowing for faster training times and more efficient use of resources.

Model deployment: TensorFlow allows for easy deployment of models to a variety of platforms, including mobile devices and the web.

Visualization tools: TensorFlow provides a range of visualization tools for analyzing model performance, including TensorBoard, which allows for real-time visualization of model training and performance.

5.3 PYCHARM

PyCharm is an integrated development environment (IDE) for Python programming language, developed by JetBrains. PyCharm provides features such as code completion, debugging, code analysis, refactoring, version control integration, and more to help developers write, test, and debug their Python code efficiently. PyCharm is available in two editions: Community Edition (CE) and Professional Edition (PE). The Community Edition is a free, open-source version of the IDE that provides basic functionality for Python development. The Professional Edition is a paid version of the IDE that provides advanced features such as remote development, web development, scientific tools, database tools, and more. PyCharm is available for Windows, macOS, and Linux operating systems. It supports Python versions 2.7, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, and 3.10.

Features:

- Intelligent code completion
- Syntax highlighting
- Code inspection
- Code navigation and search
- Debugging

- Testing
- Version control integration
- Web development support
- Scientific tools support
- Database tools support

Integration with other JetBrains tools

PyCharm's code completion feature can help speed up development by automatically suggesting code based on context and previously written code. It also includes a debugger that allows developers to step through code, set breakpoints, and inspect variables. PyCharm has integration with version control systems like Git, Mercurial, and Subversion. It also supports virtual environments, which allow developers to manage different Python installations and packages in isolated environments. The IDE also has features specifically geared towards web development, such as support for popular web frameworks like Django, Flask, and Pyramid. It includes tools for debugging, testing, and profiling web applications. PyCharm also provides scientific tools for data analysis, visualization, and scientific computing, such as support for NumPy, SciPy, and matplotlib. It also includes tools for working with databases, such as PostgreSQL, MySQL, and Oracle. Overall, PyCharm is a powerful and feature-rich IDE that can greatly increase productivity for Python developers.

Customization:

PyCharm allows developers to customize the IDE to their liking. Users can change the color scheme, fonts, and other settings to make the IDE more comfortable to use. PyCharm also supports plugins, which allow developers to extend the IDE with additional features.

Collaboration:

PyCharm makes it easy for developers to collaborate on projects. It supports integration with popular collaboration tools such as GitHub, Bitbucket, and GitLab. It also includes features for code reviews, task management, and team communication.

Education:

PyCharm provides a learning environment for Python programming language. PyCharm Edu is a free, open-source edition of PyCharm that includes interactive courses and tutorials for learning Python. It provides an easy-to-use interface for beginners and includes features such as code highlighting, autocompletion, and error highlighting.

Support:

PyCharm has an active community of users who provide support through forums and social media. JetBrains also provides comprehensive documentation, tutorials, and training courses for PyCharm. For users who need more personalized support, JetBrains offers a paid support plan that includes email and phone support.

Pricing:

PyCharm Community Edition is free and open-source. PyCharm Professional Edition requires a paid license, but offers a 30-day free trial. JetBrains also offers a subscription-based pricing model that includes access to all JetBrains IDEs and tools.

Integrations:

PyCharm integrates with a wide range of tools and technologies commonly used in Python development. It supports popular Python web frameworks like Flask, Django, Pyramid, and web2py. It also integrates with tools for scientific computing like NumPy, SciPy, and pandas. PyCharm also supports popular front-end technologies such as HTML, CSS, and JavaScript.

Performance:

PyCharm is known for its fast and reliable performance. It uses a combination of static analysis, incremental compilation, and intelligent caching to provide fast code completion and navigation. PyCharm also has a memory profiler that helps identify and optimize memory usage in Python applications.

Ease of Use:

PyCharm provides an intuitive and easy-to-use interface for developers. It has a well-organized menu structure, clear icons, and easy-to-navigate tabs. PyCharm also provides a variety of keyboard shortcuts and customizable keymaps that allow users to work efficiently without constantly switching between the mouse and keyboard.

Community:

PyCharm has a large and active community of developers who contribute to the development of the IDE. The PyCharm Community Edition is open-source, which means that anyone can contribute to its development. The PyCharm user community is also active in providing support, tips, and tutorials through forums, blogs, and social media.

5.4 MY SQL

MySQL is the world's most used open source relational database management system (RDBMS) as of 2008 that run as a server providing multi-user access to a number of databases. The MySQL development project has made its source code available under the terms of the GNU General Public License, as well as under a variety of proprietary agreements. MySQL was owned and sponsored by a single for-profit firm, the Swedish company MySQL AB, now owned by Oracle Corporation.

MySQL is a popular choice of database for use in web applications, and is a central component of the widely used LAMP open source web application

software stack—LAMP is an acronym for "Linux, Apache, MySQL, Perl/PHP/Python." Free-software-open source projects that require a full-featured database management system often use MySQL. For commercial use, several paid editions are available, and offer additional functionality. Applications which use MySQL databases include: TYPO3, Joomla, Word Press, phpBB, MyBB, Drupal and other software built on the LAMP software stack. MySQL is also used in many high-profile, large-scale World Wide Web products, including Wikipedia, Google(though not for searches), ImagebookTwitter, Flickr, Nokia.com, and YouTube.

Inter images

MySQL is primarily an RDBMS and ships with no GUI tools to administer MySQL databases or manage data contained within the databases. Users may use the included command line tools, or use MySQL "front-ends", desktop software and web applications that create and manage MySQL databases, build database structures, back up data, inspect status, and work with data records. The official set of MySQL front-end tools, MySQL Workbench is actively developed by Oracle, and is freely available for use.

Graphical

The official MySQL Workbench is a free integrated environment developed by MySQL AB that enables users to graphically administer MySQL databases and visually design database structures. MySQL Workbench replaces the previous package of software, MySQL GUI Tools. Similar to other third-party packages, but still considered the authoritative MySQL frontend, MySQL Workbench lets users manage database design & modeling, SQL development (replacing MySQL Query Browser) and Database administration (replacing MySQL Administrator). MySQL Workbench is available in two editions, the regular free and open source Community Edition which may be downloaded from the MySQL website.

CHAPTER 6

SYSTEM DESIGN

6.1 SYSTEM ARCHITECTURE

The architecture diagram illustrates an intelligent traffic light control system designed to prioritize ambulance passage using computer vision and deep learning techniques. It integrates YOLO-based object detection for identifying ambulances from real-time vehicle images and a CNN model for recognizing ambulance siren sounds from collected audio datasets. Image datasets are sourced from Roboflow, and sound datasets from Kaggle, which are then used to train respective models. A camera and microphone capture live traffic images and sounds, converting them into vector datasets for classification. Once an ambulance is detected, the system communicates with traffic signals to optimize their timing, ensuring an uninterrupted passage. This framework significantly enhances emergency response times by leveraging AI-based real-time ambulance recognition and traffic light adjustments, effectively reducing congestion delays in urban environments. The below Fig. No 6.1 describes about the overall architecture of the system.

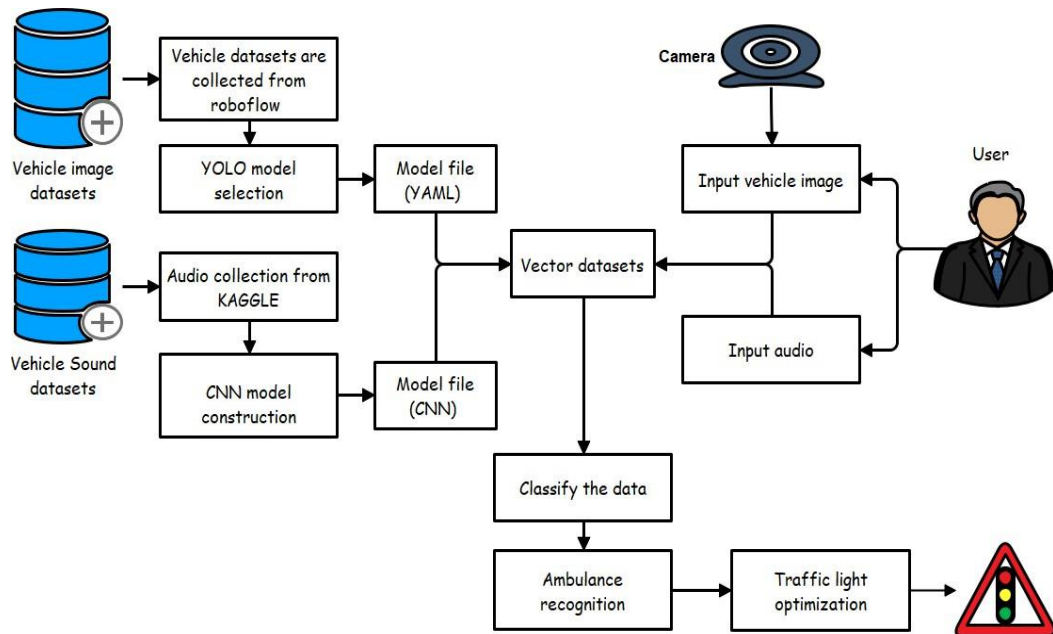


Fig. No. 6.1 System Architecture

6.2 DATA FLOW DIAGRAM

A data flow diagram shows how data is processed within a system based on inputs and outputs. Visual symbols are used to represent the flow of information, data sources and destinations, and where data is stored. Data flow diagrams are often used as a first step toward redesigning a system. They provide a graphical representation of a system at any level of detail, creating an easy-to-understand picture of what the system does. A general overview of a system is represented with a context diagram, also known as a level 0 DFD, which shows a system as a single process. A level 1 diagram provides greater detail, focusing on a system's main functions. Diagrams that are level 2 or higher illustrate a system's functioning with increasing detail. It's rare for a DFD to go beyond level 2 because of the increasing complexity, which makes it less effective as a communication tool.

LEVEL 0

The Level 0 DFD shows how the system is divided into 'sub-systems' (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. It also identifies internal data stores that must be present in order for the system to do its job. The below Fig. No. 6.2 shows the flow of data between the various parts of the system.

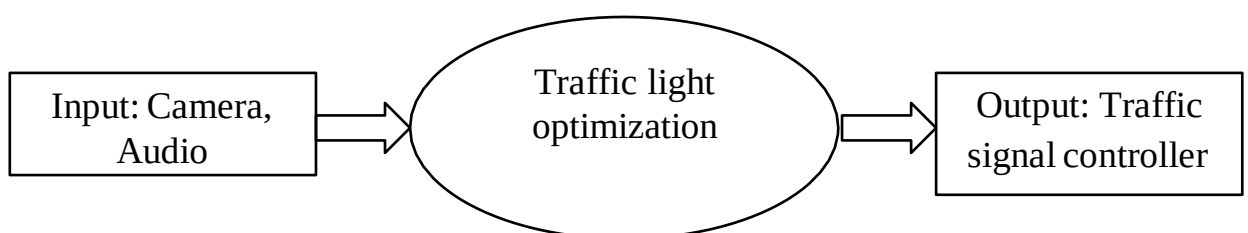


Fig. No. 6.2 Data flow diagram level 0

LEVEL 1

The next stage is to create the Level 1 Data Flow Diagram. This highlights the main functions carried out by the system. As a rule, to describe the system was using between two and seven functions - two being a simple system and seven being a complicated system. Thus the Fig. No 6.3 enables us to keep the model manageable on screen or paper.

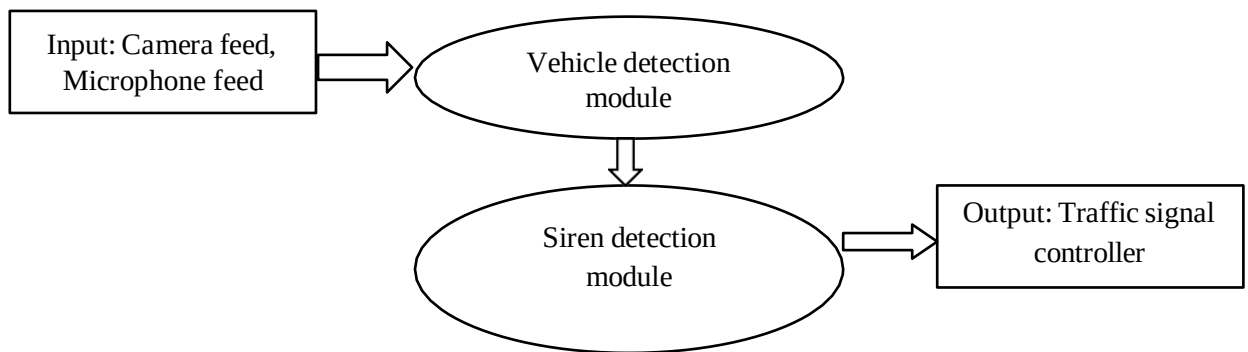


Fig. No. 6.3 Data flow diagram Level 1

LEVEL 2

A Data Flow Diagram (DFD) tracks processes and their data paths within the business or system boundary under investigation. A DFD defines each domain boundary and illustrates the logical movement and transformation of data within the defined boundary. The Fig. No 6.4 shows 'what' input data enters the domain, 'what' logical processes the domain applies to that data, and 'what' output data leaves the domain. Essentially, a DFD is a tool for process modeling and one of the oldest.

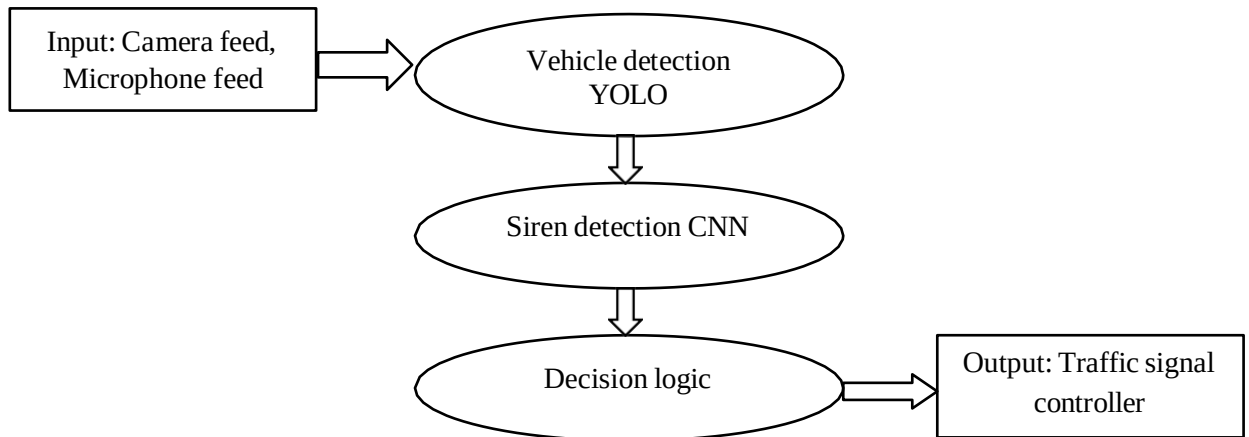


Fig. No. 6.4 Data flow diagram level 2

6.3 UML DIAGRAMS

A UML (Unified Modeling Language) diagram is a standardized visual representation used to model the design and structure of software systems. It helps developers, designers, and stakeholders to understand how different parts of a system interact, making complex systems easier to comprehend.



Fig. No. 6.5 Use case diagram

In its most basic form, the below Fig. No 6.5 is a depiction of a user's interaction with the system that illustrates the connection between the user and the many use cases that the user is involved in. A "system" in this sense refers to something that is being created or run, like a website. The "actors" are individuals or groups functioning inside the system in designated roles.

6.3.2 Class Diagram

The class diagram defines the Unified Modeling Language (UML), is a kind of static structural diagram that illustrates a system's classes, properties, functions, and interactions between objects. The fundamental component of object-oriented modeling is the class diagram. The Fig. No 6.6 is utilized for both technical modeling which converts the models into computer code and general conceptual modeling of the applications systematic.

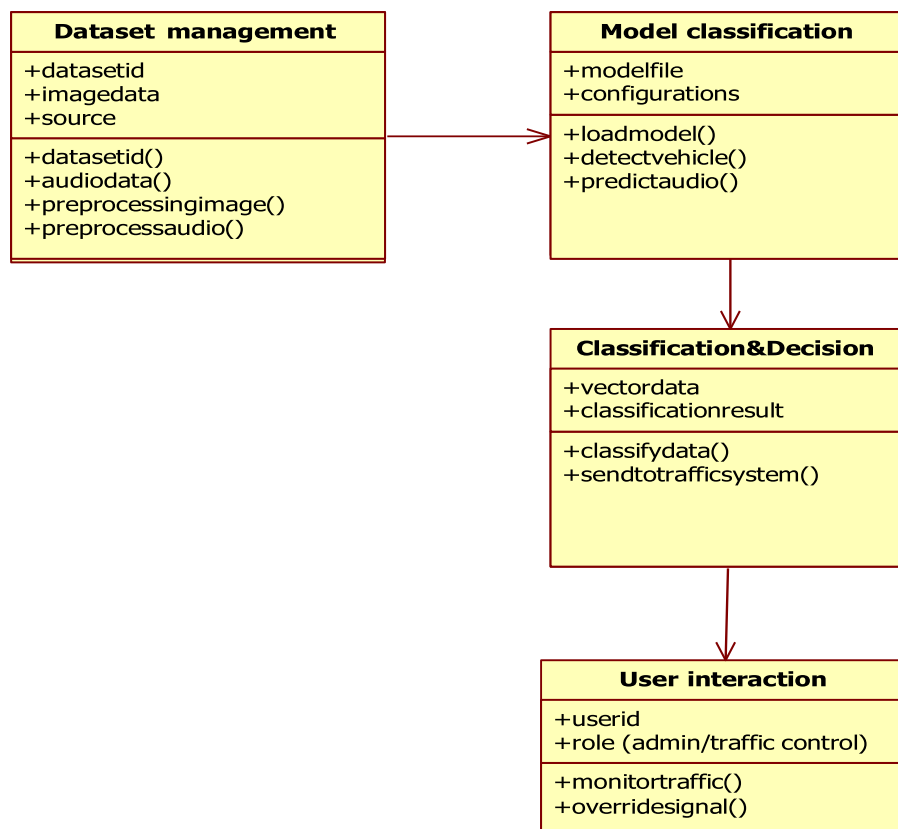


Fig. No. 6.6 Class diagram

6.3.3 Sequence Diagram

An object's interactions are arranged chronologically in a sequence diagram as shown in the Fig. no 6.7 are classes and objects that are a part of the scenario as well as the messages that are passed between the objects in order for the scenario to work. Sequence diagrams are commonly linked to the realizations of use cases in the Logical View of the system that is being developed. Event diagrams or event scenarios are other names for sequence diagrams.

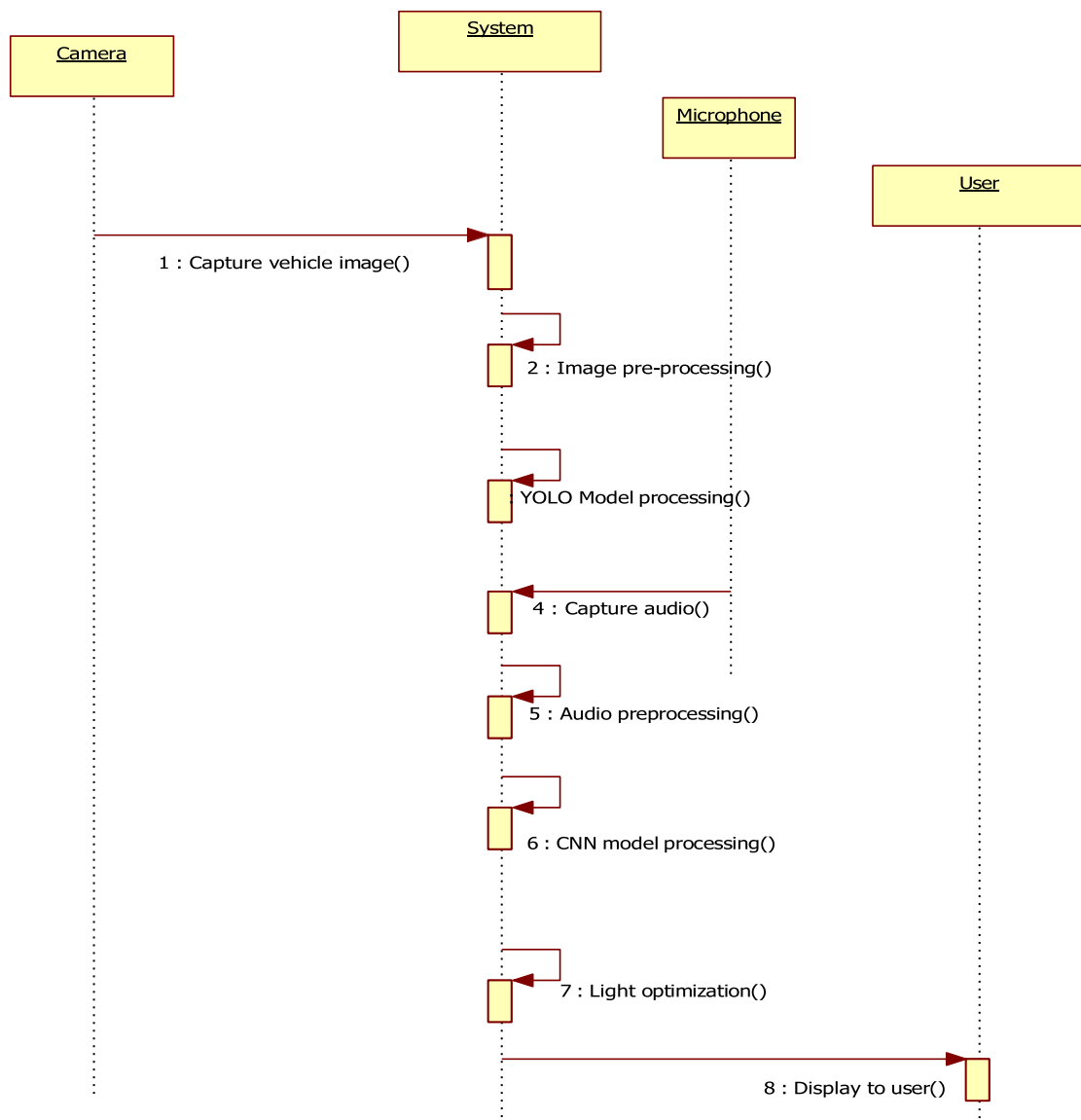


Fig. No. 6.7 Sequence diagram

6.3.4 ACTIVITY DIAGRAM

An Activity Diagram is a type of Unified Modeling Language (UML) behavioral diagram that visually represents the flow of activities or operations in a system or process. The Fig. No 6.8 is primarily used to model the dynamic aspects of a system, showing the sequence of steps, decisions, and parallel processes that occur.

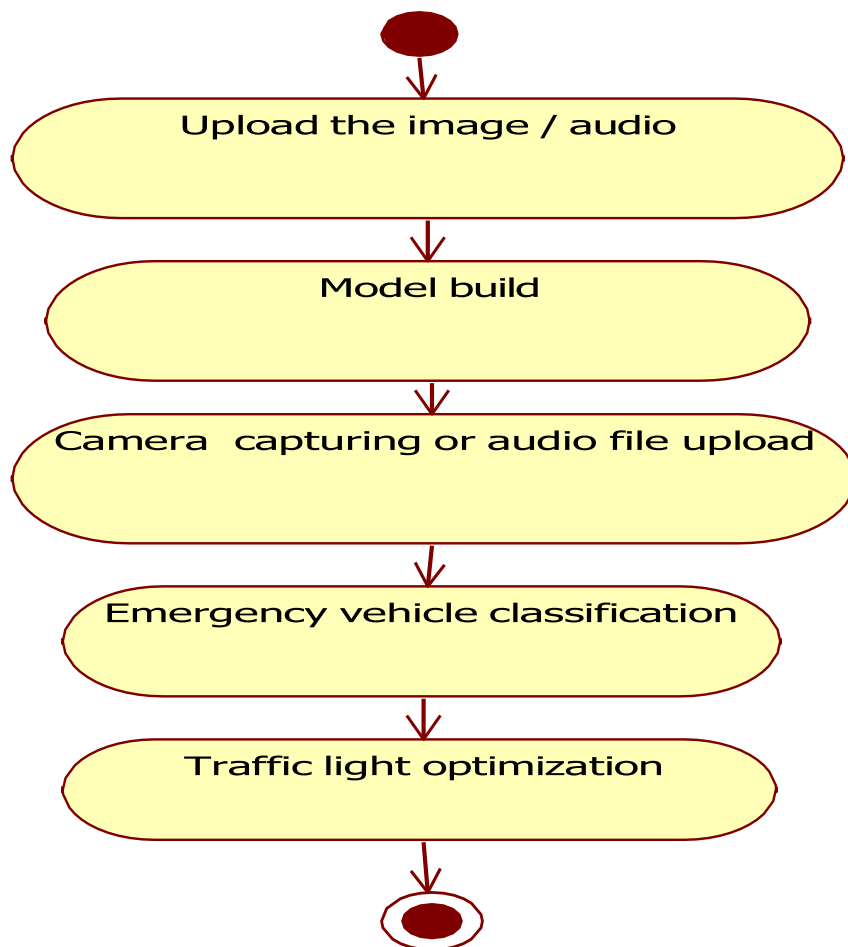


Fig. No. 6.8 Activity diagram

CHAPTER 7

PROJECT DESCRIPTION

7.1 MODULES DESCRIPTION

- DATASETS COLLECTION
- VEHICLE IMAGE
- AUDIO DATASETS
- MODEL CONSTRUCTION
- REAL TIME INTERFACE
- CLASSIFY THE VEHICLES

7.1.1 DATASETS COLLECTION

The dataset collection module is responsible for gathering relevant vehicle images and audio data required for training the ambulance detection system. The vehicle images are collected from Roboflow, which provides a wide range of annotated datasets for training deep learning models. Similarly, emergency vehicle siren audio datasets are sourced from Kaggle, ensuring diverse sound variations are considered for accurate classification. The collected datasets undergo preprocessing, such as noise reduction, data augmentation, and normalization, to improve model performance. This module ensures the availability of high-quality datasets for YOLO-based image recognition and CNN-based audio classification, forming the foundation for real-time ambulance detection.

In this module, we collect vehicle images and audio to train the database, enabling classification in subsequent modules.

- **VEHICLE IMAGE**

- Collect images of different types of emergency vehicles, including ambulances, fire trucks, police cars, and rescue vehicles.
- These images should showcase vehicles from various angles, in different lighting conditions, and with varying levels of background clutter.

- **AUDIO DATASETS**

- Record audio samples of ambulance sirens at different distances, volumes, and frequencies.
- Include variations such as continuous sirens, intermittent sirens, and modulated sirens.

7.1.2 VEHICLE IMAGE

The vehicle image module focuses on processing and analyzing vehicle images captured in real-time through cameras installed at traffic intersections. The images undergo preprocessing, including normalization and resizing techniques, before being fed into a YOLO (You Only Look Once) model for detection. YOLO is a real-time object detection system that identifies ambulances among various vehicles by recognizing their unique features, such as shape, size, and color patterns. The YOLOv5 architecture is employed due to its speed and accuracy in vehicle detection. Once the system classifies a vehicle as an ambulance, the information is sent to the traffic light control system, which optimizes signals to allow the emergency vehicle to pass without obstruction. This module ensures accurate ambulance detection in complex traffic scenarios, enhancing emergency response efficiency.

7.1.3 AUDIO DATASETS

The audio dataset module deals with the collection, preprocessing, and classification of emergency vehicle siren sounds. CNN (Convolutional Neural Network) is used to extract key audio features such as frequency, amplitude, and pitch, allowing the system to distinguish ambulance sirens from general background noise. Kaggle datasets provide various emergency sirens, including those from fire trucks and police cars, enabling a diverse learning dataset. Spectrogram conversion is applied to the audio data, transforming it into a visual representation that the CNN model can analyze effectively. This module enhances the system's capability to detect ambulances in visually obstructed conditions, such as dense traffic, fog, or night-time, thereby improving accuracy in emergency vehicle identification.

7.1.4 MODEL CONSTRUCTION

The model construction module focuses on building deep learning architectures for accurate vehicle detection and classification. Two models are developed:

YOLOv5 for image-based ambulance detection

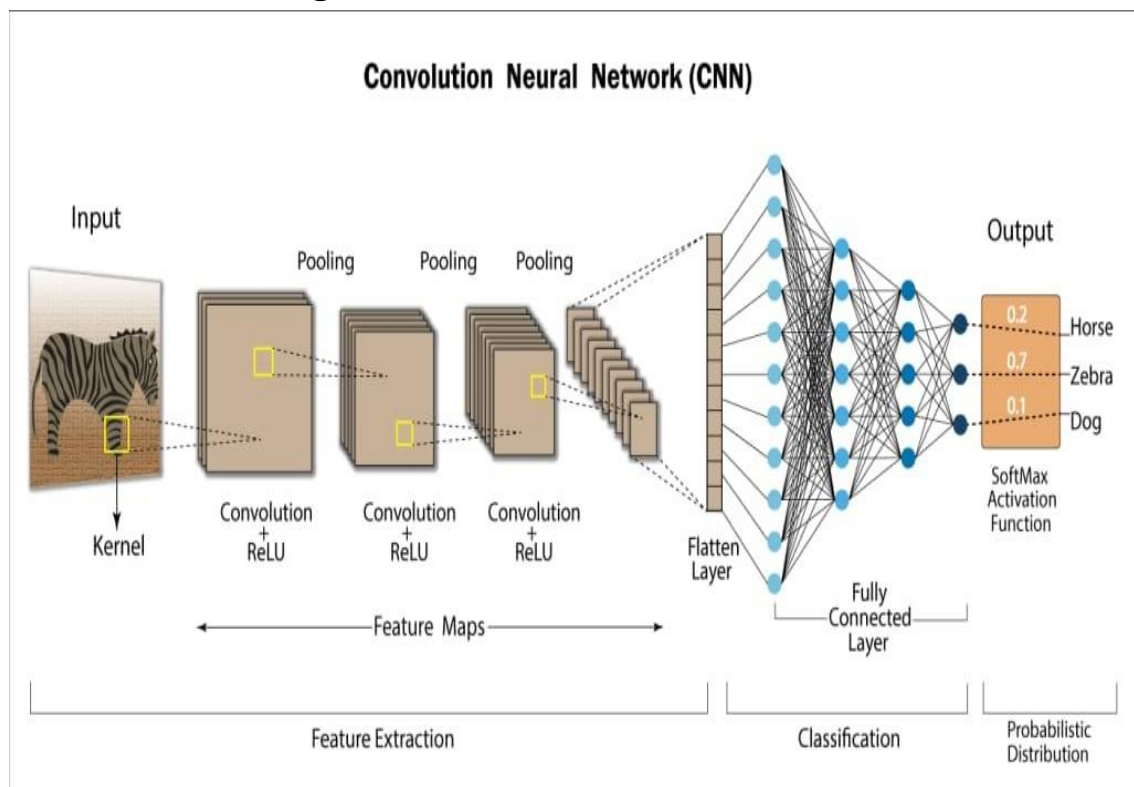
CNN for audio-based siren recognition

The YOLOv5 model uses convolutional layers with Darknet-based architecture, detecting vehicles in real-time with high precision. The model is trained on annotated datasets containing emergency vehicles and background objects to minimize false detections.

YOLO (You Only Look Once) is a real-time object detection algorithm used in the image-based process for ambulance detection. Unlike traditional methods that use region proposal-based approaches, YOLO treats object detection as a single regression problem, predicting bounding boxes and class probabilities directly from full images in one evaluation. The model divides

the input image into a grid and assigns each cell the responsibility of detecting objects within it. The below fig. no 7.1 describes the YOLO's convolutional layers extract essential features such as shape, color, and spatial details, enabling fast and accurate identification of ambulances among other vehicles in complex traffic environments. By leveraging pre-trained weights and fine-tuning on emergency vehicle datasets, YOLO enhances the system's efficiency, ensuring quick response times for real-time traffic light adjustments.

Fig. No. 7.1 Convolution neural network



7.1 YOLO ARCHITECTURE

The CNN model extracts spectrogram features from ambulance siren sounds. The architecture consists of convolutional layers, max-pooling layers, and fully connected layers, optimizing siren classification. The training process uses backpropagation and gradient descent to refine weights and improve classification accuracy. By integrating both models, the system

achieves dual-modality ambulance detection, significantly reducing response time in emergencies. A convolutional neural network is a feed-forward network with the ability of extracting topological properties from the input image. It extracts features from the raw image and then a classifier classifies extracted features. CNNs are invariance to distortions and simple geometric transformations like translation, scaling, rotation and squeezing. Convolutional Neural Networks combine three architectural ideas to ensure some degree of shift, scale, and distortion invariance: local receptive fields, shared weights, and spatial or temporal sub-sampling. The network is usually trained like a standard neural network by back propagation. A convolutional layer is used to extract features from local receptive fields in the preceding layer.

ADVANTAGES :

In a network with a 5×5 convolution kernel each unit has 25 inputs connected to a 5×5 area in the previous layer, which is the local receptive field. A trainable weight is assigned to each connection, but all units of one feature map share the same weights. This feature which allows reducing the number of trainable parameters is called weight sharing technique and is applied in all CNN layer. With local receptive fields, elementary visual features including edges can be extracted by neurons. To extract the same visual feature, neurons at different locations can share the same connection structure with the same weights. The output of such a set of neurons is a feature map. This operation is the same as a convolution of the input image with a small size kernel. Multiple feature maps can be applied to extract multiple visual features across the image. Subsampling is used to reduce the resolution of the feature map, and hence reduce the sensitivity of the output to shifts and distortions. In our proposed CNN structure, and each feature has $n = 3$ dimensions as shown in the below fig. No 7.2.

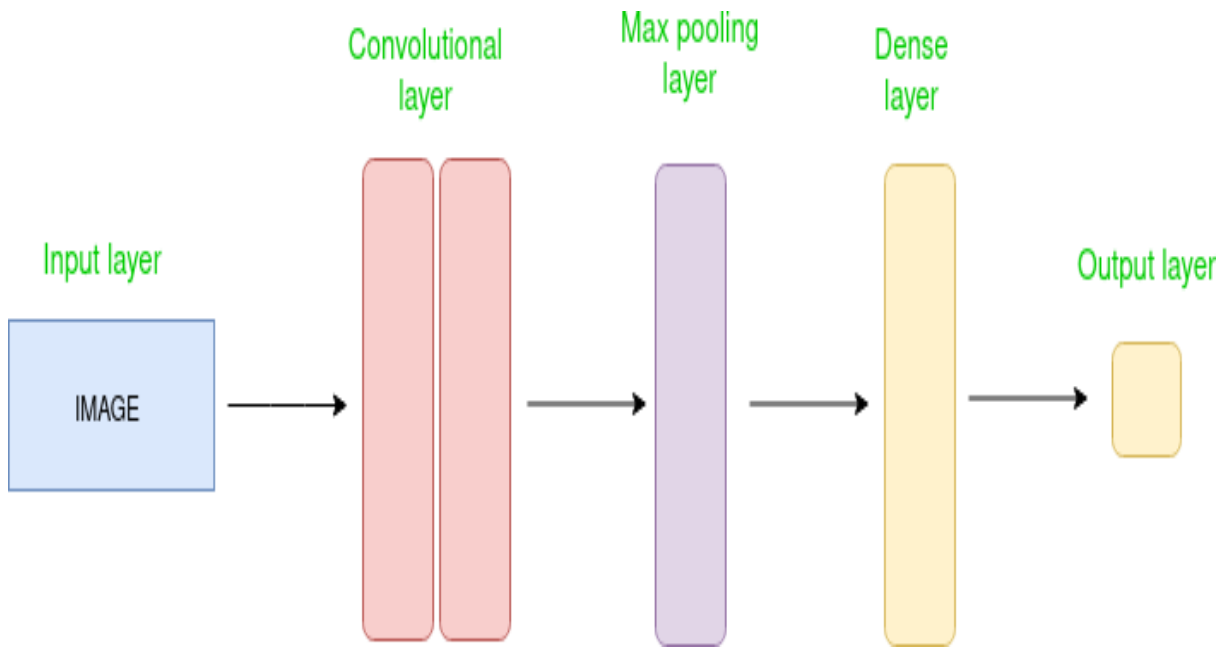


Fig.No 7.2 CNN ARCHITECTURE

7.1.5 REAL-TIME INTERFACE

The real-time interface module provides continuous monitoring and control over traffic light optimization based on ambulance detection. This module consists of computer vision-based vehicle recognition and audio signal processing. Cameras at traffic signals capture vehicle movement, while microphones detect ambulance sirens. The YOLO-based model processes images in real-time, identifying emergency vehicles. Simultaneously, the CNN model classifies detected siren sounds to confirm an approaching ambulance. Once detection is verified, the system automatically adjusts traffic signals to green in the ambulance's direction and red in other directions, ensuring an unobstructed route for emergency vehicles. Additionally, the real-time interface allows system operators to monitor live data, override controls

if needed, and update detection models periodically for continuous improvement.

7.1.6 CLASSIFY THE VEHICLES

The vehicle classification module ensures that the trained model is tested and evaluated for real-time deployment. The YOLO model classifies vehicles into categories such as ambulances, fire trucks, police cars, and general traffic. Data augmentation techniques (random rotations, brightness adjustments) are applied to improve model robustness. The system uses transfer learning to fine-tune pre-trained CNN models like ResNet or MobileNet, adapting them for specific vehicle classification tasks. Evaluation metrics such as accuracy, precision, recall, and F1-score are computed to validate model performance. Misclassified cases are analyzed to refine detection accuracy further. Finally, the classified vehicle data is integrated with traffic light optimization, allowing emergency vehicles to receive priority passage through congested intersections.

CHAPTER 8

SYSTEM TESTING

8.1 TESTING PROCESS

Testing can never completely identify all the defects within software. Instead, it furnishes a criticism or comparison that compares the state and behaviour of the product against oracles principles or mechanisms by which someone might recognize a problem. These oracles may include (but are not limited to) specifications, contracts, comparable products, past versions of the same product, inferences about intended or expected purpose, user or customer expectations, relevant standards, applicable laws, or other criteria. A primary purpose of testing is to detect software failures so that defects may be discovered and corrected.

Testing cannot establish that a product functions properly under all conditions but can only establish that it does not function properly under specific conditions. The scope of software testing often includes examination of code as well as execution of that code in various environments and conditions as well as examining the aspects of code: does it do what it is supposed to do and do what it needs to do. In the current culture of software development, a testing organization may be separate from the development team. There are various roles for testing team members. Information derived from software testing may be used to correct the process by which software is developed. Every software product has a target audience. For example, the audience for video game software is completely different from banking software. Therefore, when an organization develops or otherwise invests in a software product, it can assess whether the software product will be acceptable to its end users, its target audience, its purchasers, and other stakeholders. Software testing is the process of attempting to make this assessment

8.2 TESTING OBJECTIVES

The objective of testing is to systematically evaluate a system or application to ensure it functions as intended, meets specified requirements, and is free from defects. It involves identifying and correcting errors, validating performance, verifying functionality, and ensuring reliability, security, and usability under various conditions. Testing aims to enhance software quality, minimize risks, and ensure that the end product delivers a seamless and accurate user experience. It plays a critical role in the development lifecycle by detecting bugs early, reducing maintenance costs, and ensuring customer satisfaction.

SOFTWARE TESTING STRATEGIES

Testing involves

- Unit Testing
- Functional Testing
- Acceptance Testing
- Validation Testing

8.2.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.2.2 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.

Systems/Procedures: interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

8.2.3 ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

8.2.4 VALIDATION TESTING

At the culmination of integration testing, software is completely assembled as a package, interfacing errors have been uncovered and corrected, and a final series of software tests-validation testing begin. Validation can be defined in many ways, but a simple definition is that validation succeeds when software functions in a manner that can be reasonably expected by the customer.

8.2.5 INTEGRATION TESTING

Integration testing is any type of software testing that seeks to verify the interfaces between components against a software design. Software components may be integrated in an iterative way or all together ("big bang"). Normally the former is considered a better practice since it allows interface issues to be localised more quickly and fixed. Integration testing works to expose defects in the interfaces and interaction between integrated components (modules). Progressively larger groups of tested software components corresponding to elements of the architectural design are integrated and tested until the software works as a system.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

Efficient emergency response in urban environments is crucial to saving lives, and traffic congestion poses a significant challenge to ambulances reaching their destinations on time. This project addresses this critical issue by developing an intelligent traffic light control system that prioritizes emergency vehicles using a combination of computer vision, audio processing, and deep learning algorithms. The system integrates YOLO-based vehicle detection and CNN-based sound classification to accurately identify ambulances in real-time. Once detected, the system dynamically adjusts traffic signal timings to create a clear path for the emergency vehicle, reducing unnecessary delays and improving overall traffic management.

By leveraging advanced deep learning techniques and real-time communication, the system enhances the speed and efficiency of emergency response while ensuring compliance with traffic regulations. The proposed framework is tested across varied real-world scenarios to ensure accuracy, reliability, and effectiveness. The system significantly outperforms traditional machine learning approaches in recognizing emergency vehicles, offering higher precision and faster processing speeds. Its ability to adapt to different traffic conditions makes it a scalable and robust solution for modern urban traffic infrastructure. With continued improvements and integration with smart city initiatives, this intelligent traffic control system has the potential to revolutionize emergency response management, reducing fatalities and enhancing public safety. The implementation of such AI-driven solutions paves the way for a more efficient, responsive, and intelligent traffic ecosystem that prioritizes human lives over congestion.

9.2 FUTURE ENHANCEMENTS

- **Integration with IoT and Smart Traffic Systems:** Future improvements can involve integrating IoT-based smart sensors at traffic intersections to enhance real-time detection and decision-making. By connecting the system to a centralized cloud-based traffic control platform, authorities can receive live updates and optimize traffic flow dynamically. This would improve the coordination of emergency vehicle movement across multiple intersections, ensuring smooth and uninterrupted passage.
- **Implementation of 5G and Edge Computing for Faster Processing:** Leveraging 5G networks and edge computing can significantly reduce latency in data processing and decision-making. Instead of relying on centralized servers, processing vehicle images and siren audio at the edge (i.e., directly at the traffic signals) can enhance response time. This will allow for near-instantaneous traffic light adjustments, minimizing delays and improving emergency response efficiency.
- **Multi-Modal Emergency Vehicle Recognition:** The system can be extended to support multiple emergency vehicles, such as fire trucks and police cars, with unique prioritization rules. By training the model on a broader dataset and refining classification algorithms, the system can differentiate between various emergency vehicles and adjust traffic signals based on their urgency levels.
- **Integration with GPS and Navigation Systems:** By connecting with real-time GPS navigation systems used by emergency responders, the system can predict the arrival of an ambulance and preemptively clear traffic. This predictive approach can further reduce waiting times by synchronizing multiple traffic signals along the ambulance's route, ensuring seamless passage.

CHAPTER 10

APPENDIX

10.1 SOURCE CODE

```
from keras.models import Sequential

from keras.layers import Convolution2D

from keras.layers import MaxPooling2D

from keras.layers import Flatten

from keras.layers import Dense

from keras.models import model_from_json

import matplotlib.pyplot as plt

import warnings

warnings.filterwarnings('ignore')

batch_size = 32


from tensorflow.keras.preprocessing.image import ImageDataGenerator

# All images will be rescaled by 1./255

train_datagen = ImageDataGenerator(rescale=1/255)

# Flow training images in batches of 128 using train_datagen generator

train_generator = train_datagen.flow_from_directory(

    'DataSet/train/', # This is the source directory for training images

    target_size=(200, 200), # All images will be resized to 200 x 200

    batch_size=batch_size,
```

```

# Specify the classes explicitly

classes =
['Ambulance','Bicycle','Bus','Car','Motorcycle','Tank','Taxi','Truck'],

# Since we use categorical_crossentropy loss, we need categorical labels
class_mode='categorical')

import tensorflow as tf

model = tf.keras.models.Sequential([

    # Note the input shape is the desired size of the image 200x 200 with 3
    bytes color

    # The first convolution

    tf.keras.layers.Conv2D(16, (3,3), activation='relu', input_shape=(200, 200,
    3)),

    tf.keras.layers.MaxPooling2D(2, 2),

    # The second convolution

    tf.keras.layers.Conv2D(32, (3,3), activation='relu'),

    tf.keras.layers.MaxPooling2D(2,2),

    # The third convolution

    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),

    tf.keras.layers.MaxPooling2D(2,2),

    # The fourth convolution

    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),

    tf.keras.layers.MaxPooling2D(2,2),

```

```

    # The fifth convolution
    tf.keras.layers.Conv2D(64, (3,3), activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),

    # Flatten the results to feed into a dense layer
    tf.keras.layers.Flatten(),

    # 128 neuron in the fully-connected layer
    tf.keras.layers.Dense(128, activation='relu'),

    # 5 output neurons for 5 classes with the softmax activation
    tf.keras.layers.Dense(8, activation='softmax')
])

model.summary()

from tensorflow.keras.optimizers import RMSprop
early = tf.keras.callbacks.EarlyStopping(monitor='val_loss',patience=5)
model.compile(loss='categorical_crossentropy',
              optimizer=RMSprop(lr=0.001),
              metrics=['accuracy'])

total_sample=train_generator.n

```

```

n_epochs = 10

history = model.fit_generator(
    train_generator,
    steps_per_epoch=int(total_sample/batch_size),
    epochs=n_epochs,
    verbose=1)

model.save('model.h5')

acc = history.history['accuracy']

loss = history.history['loss']

epochs = range(1, len(acc) + 1)

# Train and validation accuracy

plt.plot(epochs, acc, 'b', label='Training accuracy')

plt.title('Training accuracy')

plt.legend()

plt.figure()

# Train and validation loss

plt.plot(epochs, loss, 'b', label='Training loss')

plt.title('Training loss')

plt.legend()

plt.show()

```

```

import sys

import pathlib

working_dir_path = pathlib.Path().absolute()

if sys.platform.startswith('win32'):

    TRAINING_FILES_PATH = str(working_dir_path) + '\\features\\'

    SAVE_DIR_PATH = str(working_dir_path) + '\\joblib_features\\'

    MODEL_DIR_PATH = str(working_dir_path) + '\\model\\'

    TESS_ORIGINAL_FOLDER_PATH = str(working_dir_path) +
    '\\sound_set_data\\'

    EXAMPLES_PATH = str(working_dir_path) + '\\examples\\'

else:

    TRAINING_FILES_PATH = str(working_dir_path) + '/features/'

    SAVE_DIR_PATH = str(working_dir_path) + '/joblib_features/'

    MODEL_DIR_PATH = str(working_dir_path) + '/model/'

    TESS_ORIGINAL_FOLDER_PATH = str(working_dir_path) +
    '/sound_set_data/'

    EXAMPLES_PATH = str(working_dir_path) + '/examples/'

```

''''''

This files creates the X and y features in joblib to be used by the predictive models.

''''''

```

import os

import time

import joblib

import librosa

import numpy as np

from config import SAVE_DIR_PATH

from config import TRAINING_FILES_PATH

class CreateFeatures:

    @staticmethod

    def features_creator(path, save_dir) -> str:

        """

        This function creates the dataset and saves both data and labels in

        two files, X.joblib and y.joblib in the joblib_features folder.

        With this method, you can persist your features and train quickly

        new machine learning models instead of reloading the features

        every time with this pipeline.

        """

        lst = []

        start_time = time.time()

        for subdir, dirs, files in os.walk(path):

            for file in files:

```

```

print(file)

try:

    # Load librosa array, obtain mfccs, store the file and the mfccs
    information in a new array

    X, sample_rate = librosa.load(os.path.join(subdir, file),
    res_type='kaiser_fast')

    mfccs = np.mean(librosa.feature.mfcc(y=X, sr=sample_rate,
    n_mfcc=40).T, axis=0)

    # The instruction below converts the labels (from 1 to 8) to a
    series from 0 to 7

    # This is because our predictor needs to start from 0 otherwise it
    will try to predict also 0.

    file = str(file[0:3])

    print(file)

    if file == 'amb':

        file = 0

    else:

        file = 1

    print(file)

arr = mfccs, file

print(arr)

lst.append(arr)

```



```

        # If the file is not valid, skip it

        except ValueError as err:

            print(err)

            continue

print("--- Data loaded. Loading time: %s seconds ---" % (time.time() -
start_time))

    # Creating X and y: zip makes a list of all the first elements, and a list of
all the second elements.

    X, y = zip(*lst)

    # Array conversion

    X, y = np.asarray(X), np.asarray(y)

    # Array shape check

print(X.shape, y.shape)

    # Preparing features dump

X_name, y_name = 'X.joblib', 'y.joblib'

joblib.dump(X, os.path.join(save_dir, X_name))

joblib.dump(y, os.path.join(save_dir, y_name))

    return "Completed"

if __name__ == '__main__':

    print('Routine started')

    FEATURES =

    CreateFeatures.features_creator(path=TRAINING_FILES_PATH,

```

```

save_dir=SAVE_DIR_PATH)

print('Routine completed.')

"""

This file can be used to try a live prediction.

"""

import keras

import librosa

import numpy as np

from config import EXAMPLES_PATH

from config import MODEL_DIR_PATH

class LivePredictions:

    """

    Main class of the application.

    """

    def __init__(self, file):

        """

        Init method is used to initialize the main parameters.

        """

        self.file = file

        self.path = MODEL_DIR_PATH + 'sound_Model.h5'

        self.loaded_model = keras.models.load_model(self.path)

```

```

def make_predictions(self):
    """
    Method to process the files and create your features.
    """
    data, sampling_rate = librosa.load(self.file)

    mfccs = np.mean(librosa.feature.mfcc(y=data, sr=sampling_rate,
n_mfcc=40).T, axis=0)

    x = np.expand_dims(mfccs, axis=2)

    x = np.expand_dims(x, axis=0)

    predictions = self.loaded_model.predict_classes(x)

    print( "Prediction is", " ", self.convert_class_to_emotion(predictions))

    @staticmethod
    def convert_class_to_emotion(pred):
        """
        Method to convert the predictions (int) into human readable strings.
        """

        label_conversion = {'0': 'ambulance',
                             '1': 'traffic'
                             }

        for key, value in label_conversion.items():
            if int(key) == pred:
                label = value

```

```

        return label

if __name__ == '__main__':

    live_prediction = LivePredictions(file=EXAMPLES_PATH + 'sound_1.wav')

    live_prediction.loaded_model.summary()

    live_prediction.make_predictions()

    live_prediction = LivePredictions(file=EXAMPLES_PATH +
'sound_401.wav')

    live_prediction.make_predictions()

    """

    Neural network train file.

    """

    import os

    import joblib

    import numpy as np

    import matplotlib.pyplot as plt

    from keras.layers import Dense

    from keras.layers import Conv1D

    from keras.layers import Flatten

    from keras.layers import Dropout

    from keras.layers import Activation

    from keras.models import Sequential

    from sklearn.metrics import confusion_matrix

```

```

from sklearn.metrics import classification_report

from sklearn.model_selection import train_test_split

from config import SAVE_DIR_PATH

from config import MODEL_DIR_PATH

class TrainModel:

    @staticmethod

    def train_neural_network(X, y) -> None:

        """

        This function trains the neural network.

        """

        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,
        random_state=42)

        x_traincnn = np.expand_dims(X_train, axis=2)

        x_testcnn = np.expand_dims(X_test, axis=2)

        print(x_traincnn.shape, x_testcnn.shape)

        model = Sequential()

        model.add(Conv1D(64, 5, padding='same',
        input_shape=(40, 1)))

        model.add(Activation('relu'))

        model.add(Dropout(0.2))

        model.add(Flatten())

        model.add(Dense(2))

```

```

model.add(Activation('softmax'))

print(model.summary)

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='rmsprop',
              metrics=['accuracy'])

cnn_history = model.fit(x_traincnn, y_train,
                        batch_size=16, epochs=50,
                        validation_data=(x_testcnn, y_test))

# Loss plotting

plt.plot(cnn_history.history['loss'])

plt.plot(cnn_history.history['val_loss'])

plt.title('model loss')

plt.ylabel('loss')

plt.xlabel('epoch')

plt.legend(['train', 'test'], loc='upper left')

plt.savefig('loss.png')

plt.close()

# Accuracy plotting

plt.plot(cnn_history.history['accuracy'])

plt.plot(cnn_history.history['val_accuracy'])

plt.title('model accuracy')

```

```

plt.ylabel('acc')

plt.xlabel('epoch')

plt.legend(['train', 'test'], loc='upper left')

plt.savefig('accuracy.png')

    predictions = model.predict_classes(x_testcnn)

new_y_test = y_test.astype(int)

    matrix = confusion_matrix(new_y_test, predictions)

print(classification_report(new_y_test, predictions))

    print(matrix)

model_name = 'sound_Model.h5'

    # Save model and weights

    if not os.path.isdir(MODEL_DIR_PATH):

os.makedirs(MODEL_DIR_PATH)

model_path = os.path.join(MODEL_DIR_PATH, model_name)

model.save(model_path)

print('Saved trained model at %s ' % model_path)

if __name__ == '__main__':

    print('Training started')

    print(SAVE_DIR_PATH + 'X.joblib')

    X = joblib.load(SAVE_DIR_PATH + 'X.joblib')

    y = joblib.load(SAVE_DIR_PATH + 'y.joblib')

```

```

    NEURAL_NET = TrainModel.train_neural_network(X=X, y=y)

from flask import Flask, render_template, flash, request, session, send_file

from flask import render_template, redirect, url_for, request

import warnings

import datetime

app = Flask(__name__)

app.config['DEBUG']

app.config['SECRET_KEY'] = '7d441f27d441f27567d441f2b6176a'

@app.route("/")

def homepage():

    return render_template('index.html')

@app.route("/Test")

def Test():

    return render_template("Test.html")

@app.route("/ImageTest")

def ImageTest():

    return render_template('ImageTest.html')

@app.route("/imgetest", methods=['GET', 'POST'])

def imgetest():

    if request.method == 'POST':

```



```

import tensorflow as tf

import numpy as np

from keras.preprocessing import image

file = request.files['fileupload']

file.save('static/upload/Test.jpg')

fname = 'static/upload/Test.jpg'


import warnings

warnings.filterwarnings('ignore')


classifierLoad = tf.keras.models.load_model('model.h5')

test_image = image.load_img('static/upload/Test.jpg', target_size=(200, 200))

test_image = np.expand_dims(test_image, axis=0)

result = classifierLoad.predict(test_image)

print(result[0][0])

res = ""

result1 = ""


if result[0][0] == 1:

    res = 'Ambulance'

    result1 = 'Emergency'

```

```
        return render_template('Test.html', result=res, gry=fname)

elif result[0][1] == 1:

    res = 'Bicycle'

    result1 = 'Normal'

elif result[0][2] == 1:

    res = 'Bus'

    result1 = 'Normal'

elif result[0][3] == 1:

    res = 'Car'

    result1 = 'Normal'

elif result[0][4] == 1:

    res = 'Motorcycle'

    result1 = 'Normal'

elif result[0][5] == 1:

    res = 'Tank'

    result1 = 'Normal'

elif result[0][6] == 1:

    res = 'Taxi'

    result1 = 'Normal'

elif result[0][7] == 1:

    res = 'Truck'
```

```

        result1 = 'Normal'

        return render_template('ImageTest.html', result=res, result1=result1,
gry=fname)

@app.route("/testsound", methods=['GET', 'POST'])
def testsound():

    if request.method == 'POST':

        file = request.files['fileupload']

        file.save('static/Out/' + file.filename)

        live_prediction = LivePredictions(file='static/Out/' + file.filename)

        live_prediction.loaded_model.summary()

        live_prediction.make_predictions()

        res = session["out"]

        gry = "

        if res == "ambulance":

            res = 'Ambulance'

            result1 = 'Emergency'

            gry = 'static/emoji/green.jpg'

        elif res == "traffic":

            res = 'Traffic'

            result1 = 'Normal'

            gry = 'static/emoji/red.jpg'

```

```

        return render_template('Test.html', result=res, result1=result1, gry=gry)

import keras

import librosa

import numpy as np

from config import EXAMPLES_PATH

from config import MODEL_DIR_PATH

class LivePredictions:

    """

    Main class of the application.

    """

    def __init__(self, file):

        """

        Init method is used to initialize the main parameters.

        """

        self.file = file

        self.path = MODEL_DIR_PATH + 'sound_Model.h5'

        self.loaded_model = keras.models.load_model(self.path)

    def make_predictions(self):

        """

        Method to process the files and create your features.

```

```

        """

        data, sampling_rate = librosa.load(self.file)

        mfccs = np.mean(librosa.feature.mfcc(y=data, sr=sampling_rate,
        n_mfcc=40).T, axis=0)

        x = np.expand_dims(mfccs, axis=2)

        x = np.expand_dims(x, axis=0)

        predictions = self.loaded_model.predict_classes(x)

        print("Prediction is", " ", self.convert_class_to_emotion(predictions))

```

```

        session["out"] = self.convert_class_to_emotion(predictions)

```

```

    @staticmethod

```

```

    def convert_class_to_emotion(pred):

```

```

        """

```

```

        Method to convert the predictions (int) into human readable strings.

```

```

        """

```

```

    label_conversion = {'0': 'ambulance',

```

```

                        '1': 'traffic',

```

```

                        }

```

```

    for key, value in label_conversion.items():

```

```

        if int(key) == pred:

            label = value

    return label

def sendmsg(targetno, message):

    import requests

    requests.post(

        "http://smsserver9.creativepoint.in/api.php?username=fantasy&password=596692&to=" + targetno + "&from=FSSMSS&message=Dear user yourmsg is "
        + message + " Sent By FSMSG
        FSSMSS&PEID=1501563800000030506&templateid=1507162882948811640")

if __name__ == '__main__':

    app.run(debug=True, use_reloader=True)

```

10.2 SCREENSHOTS

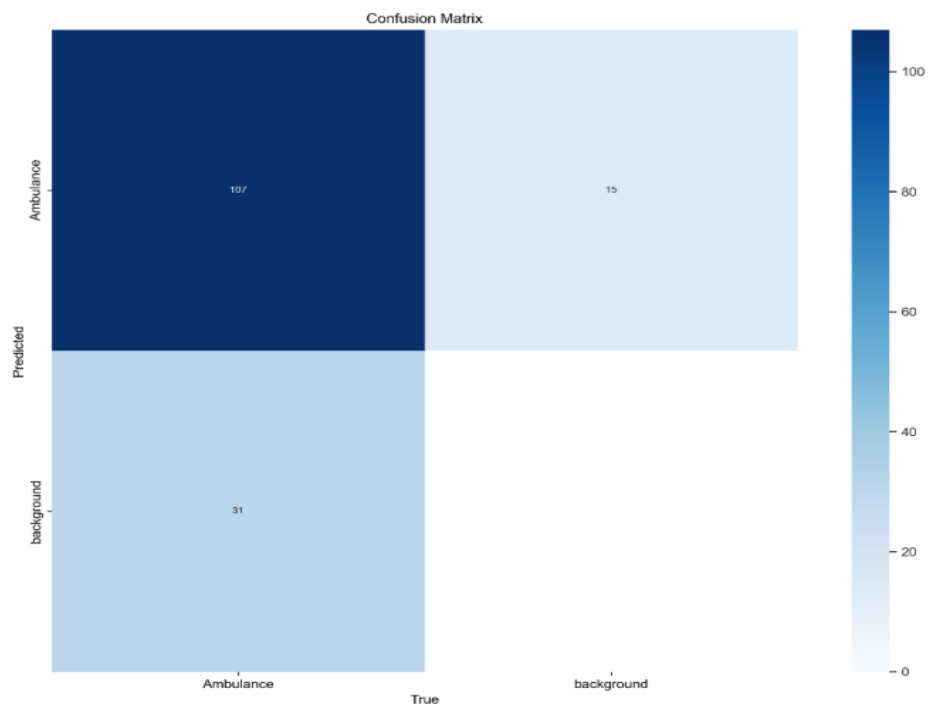


Fig. No. 10.1 Confusion Matrix

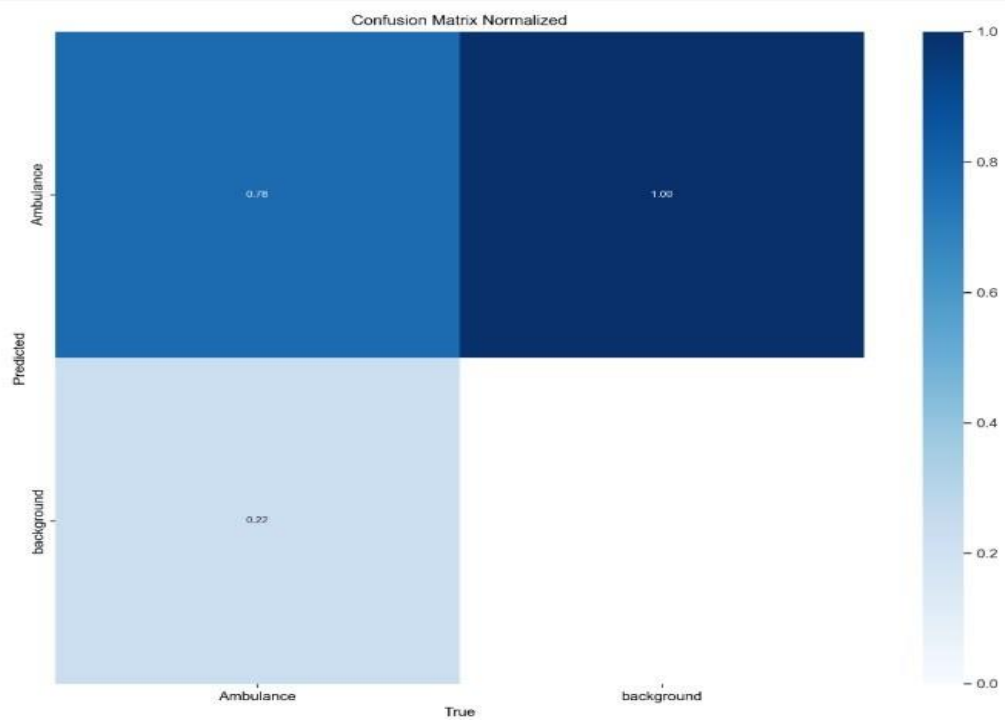


Fig. No. 10.2 Normalized confusion matrix

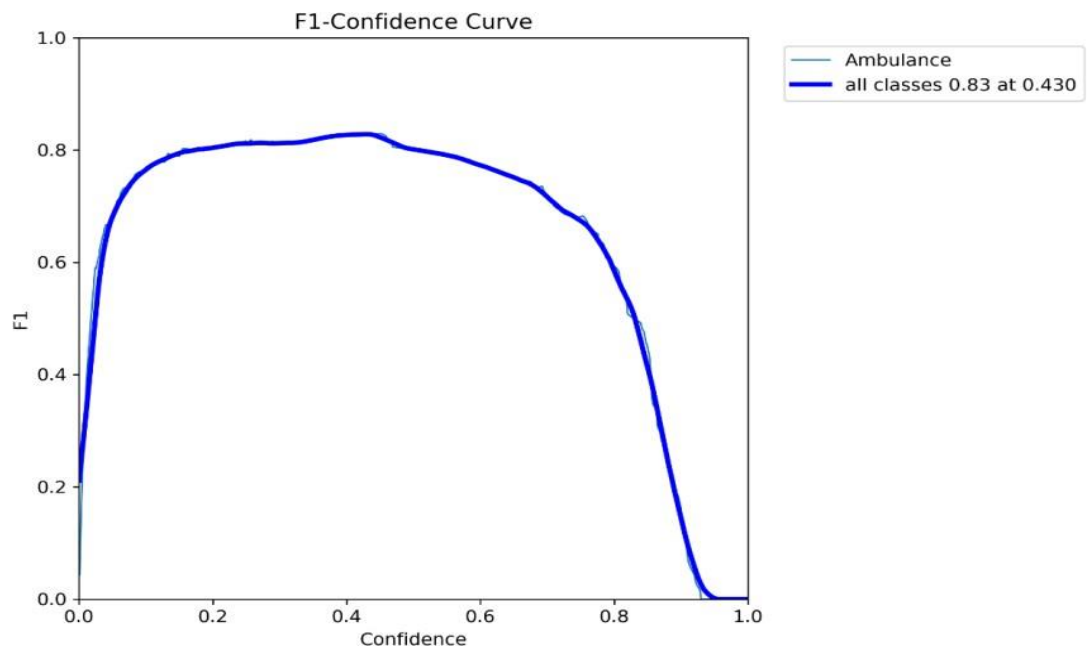


Fig. No. 10.3 Curve Graph

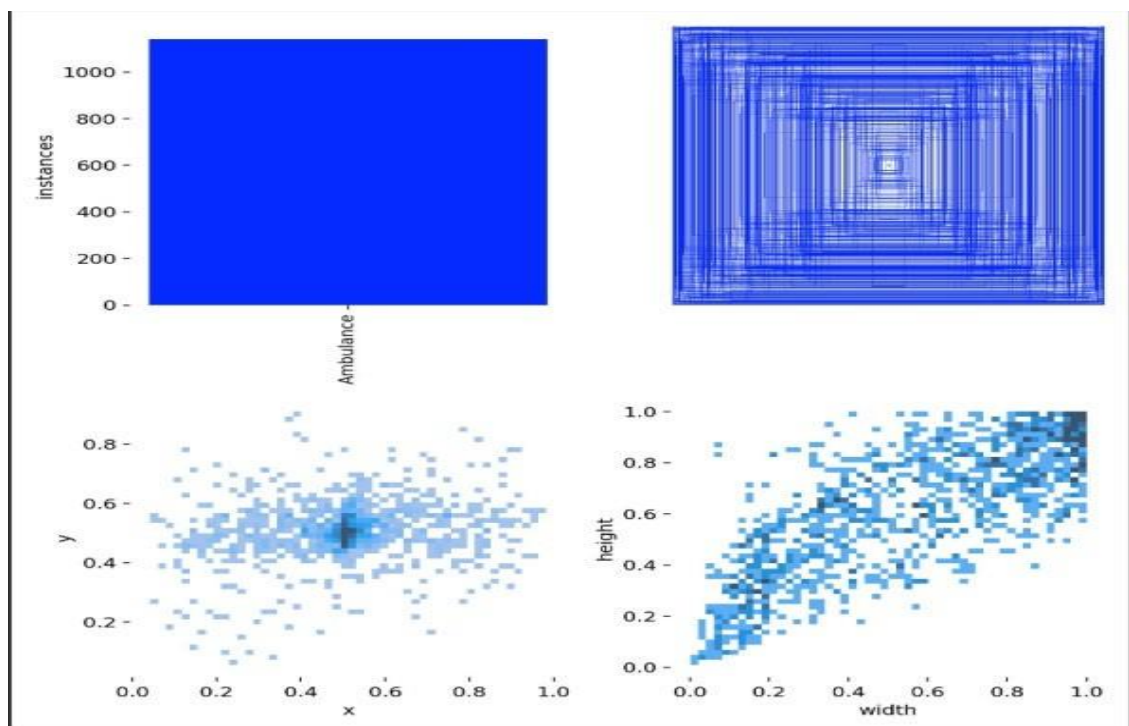


Fig. No. 10.4 Label Image

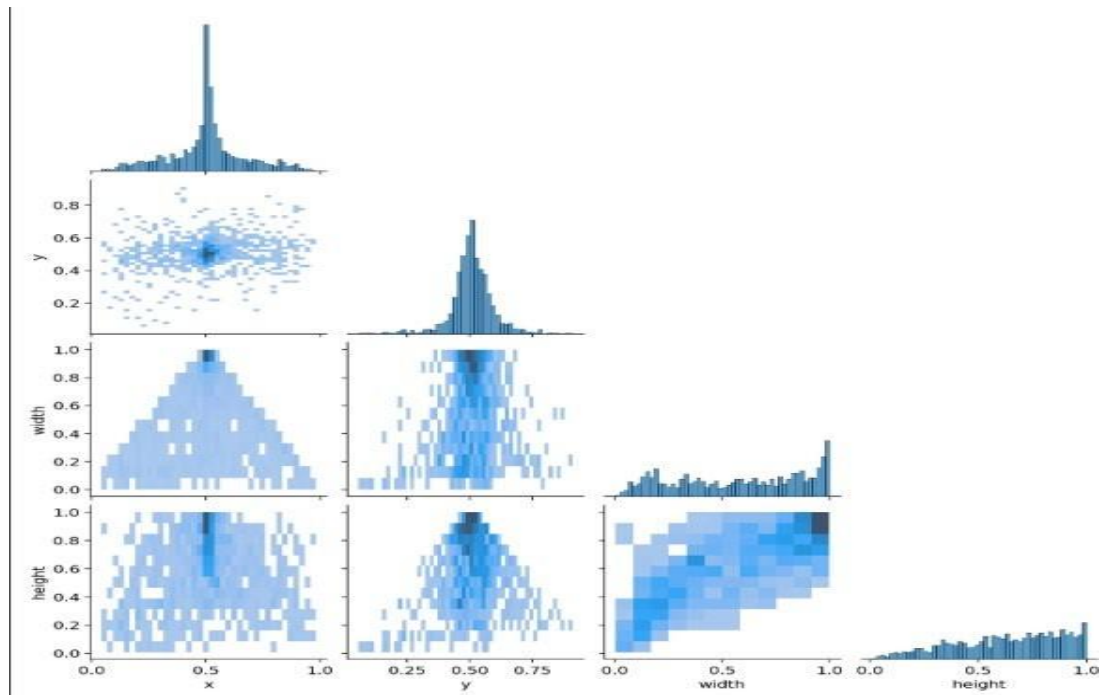


Fig. No. 10.5 Correlogram Graph

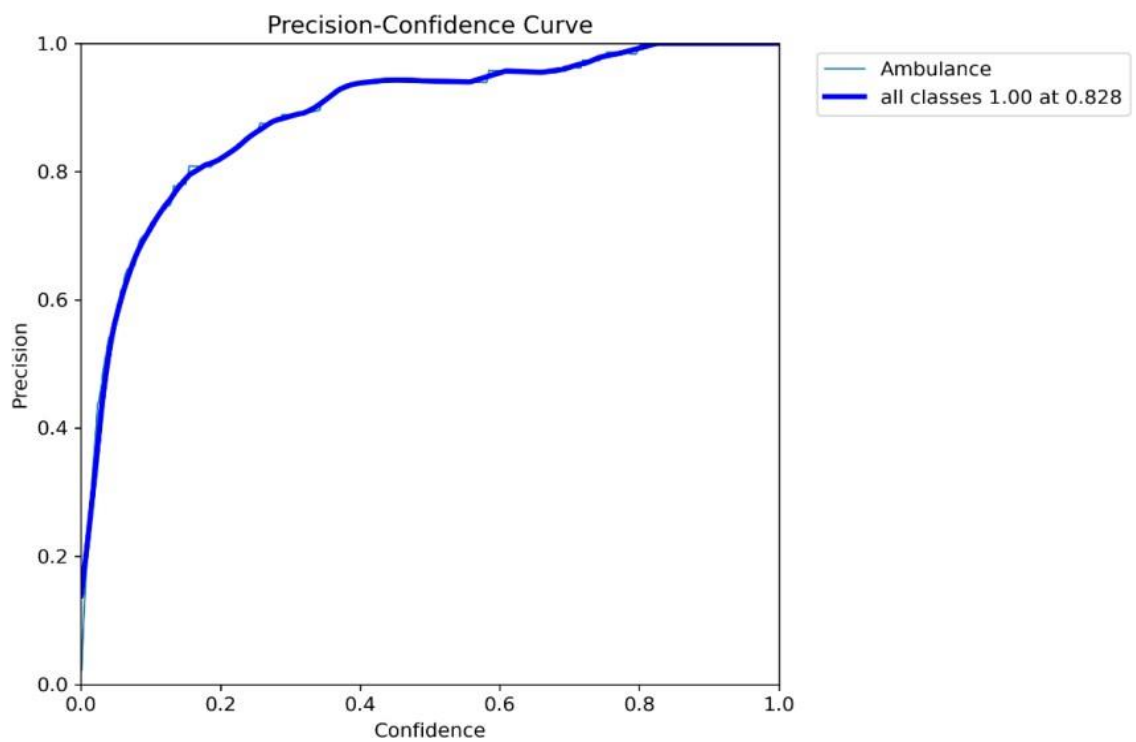


Fig. No. 10.6 Precision curve graph

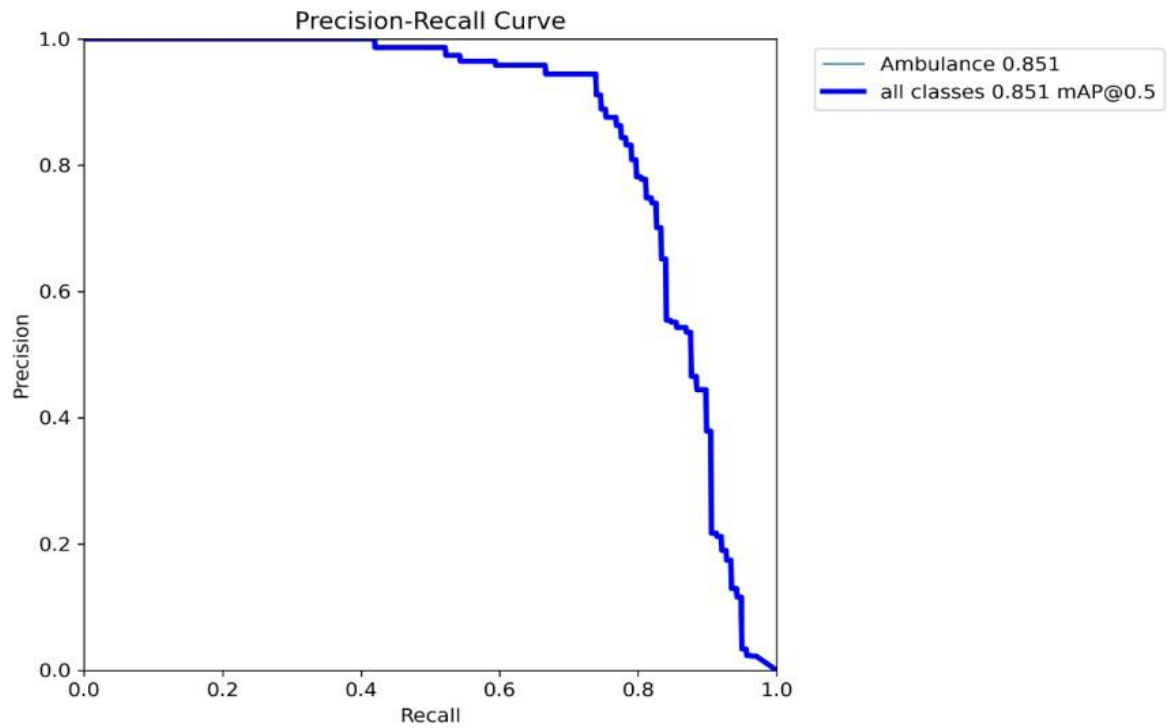


Fig No 10.7 Precision-Recall Curve

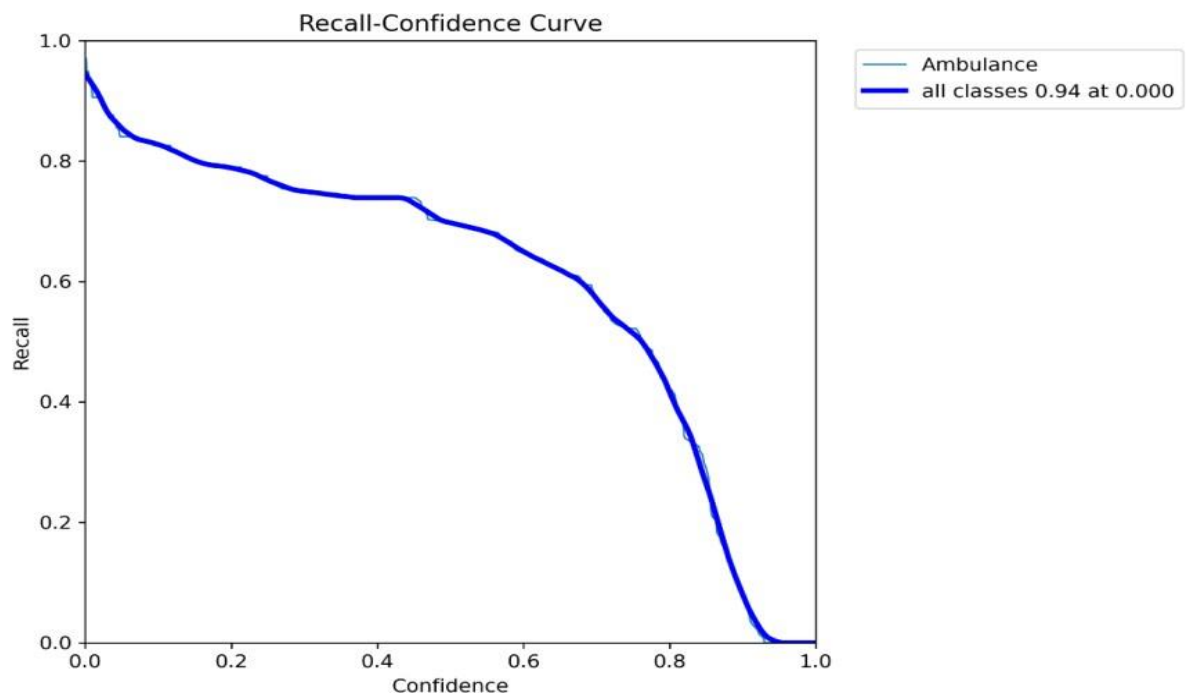


Fig. No. 10.8 Recall-Confidence curve

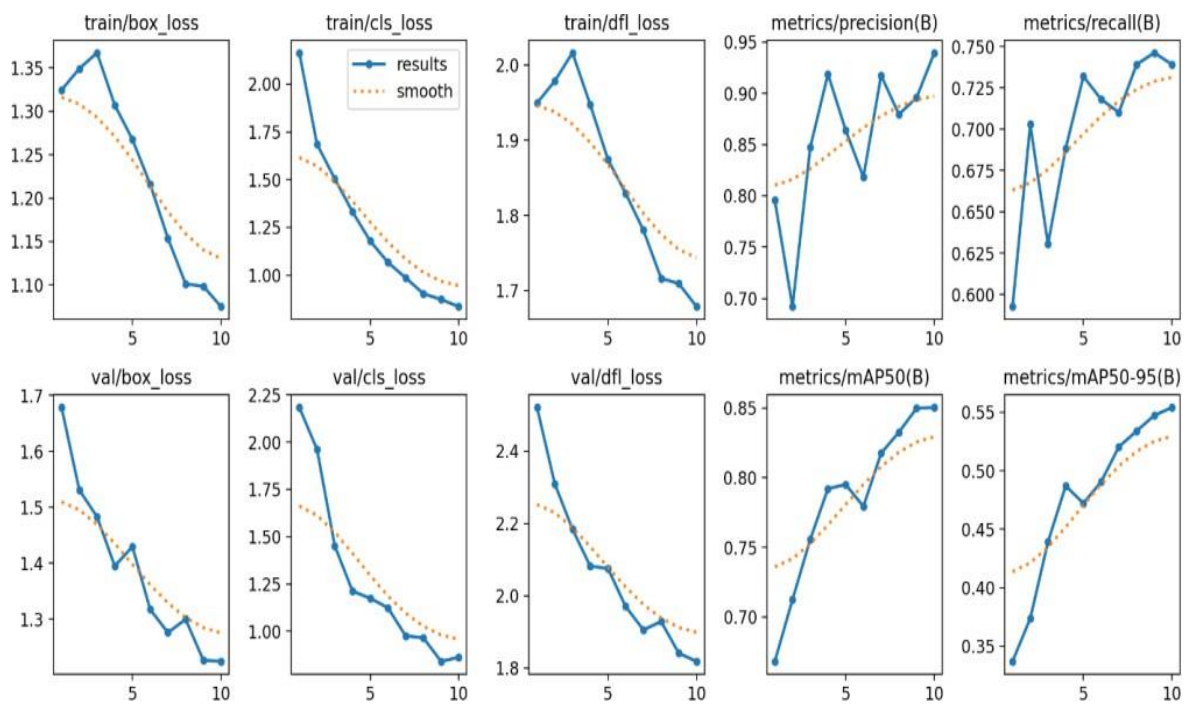


Fig. No. 10.9 Results Graph



Fig. No. 10.10 Image Detection

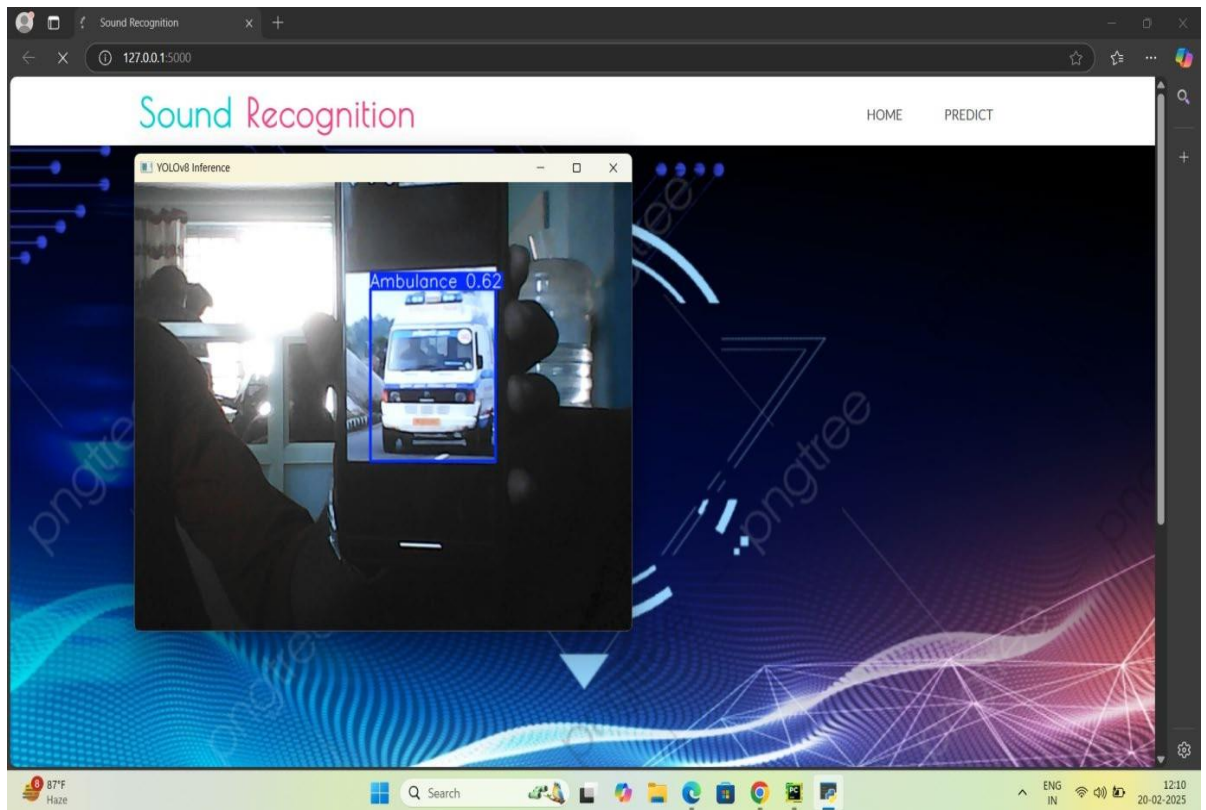


Fig. No. 10.11 Sound Recognition

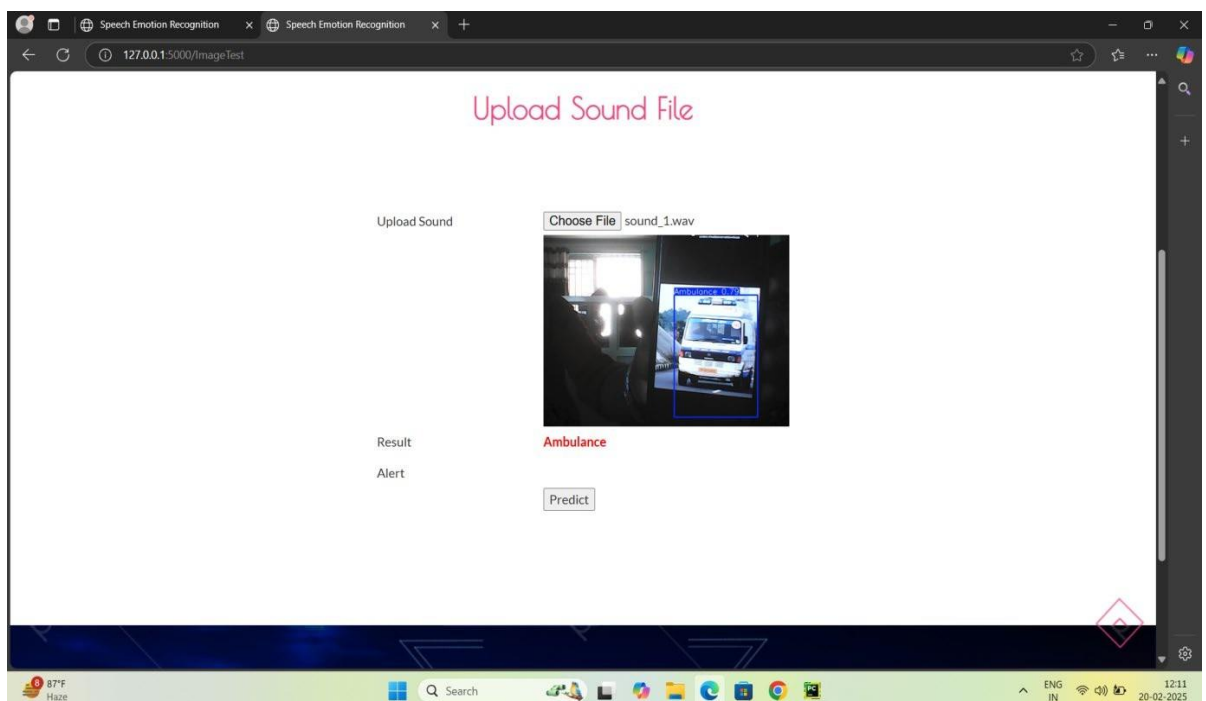


Fig. No. 10.12 Uploading Sound Recognition

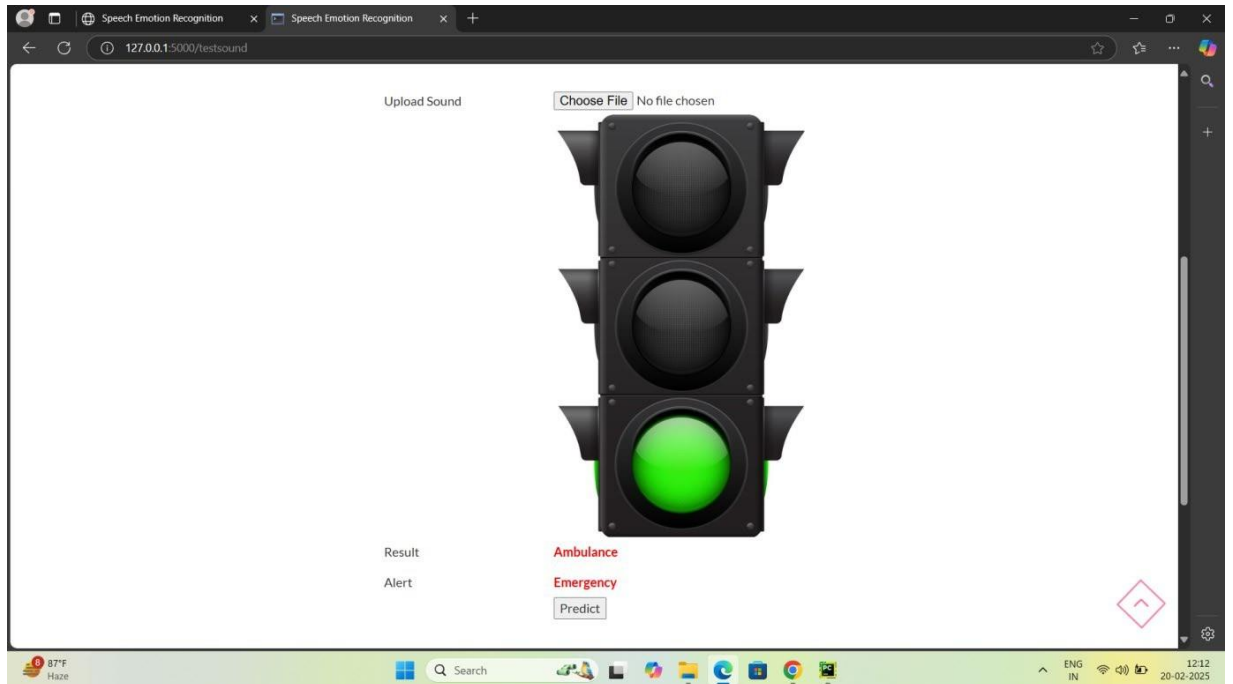


Fig. No. 10.13 Emergency Vehicle Detection

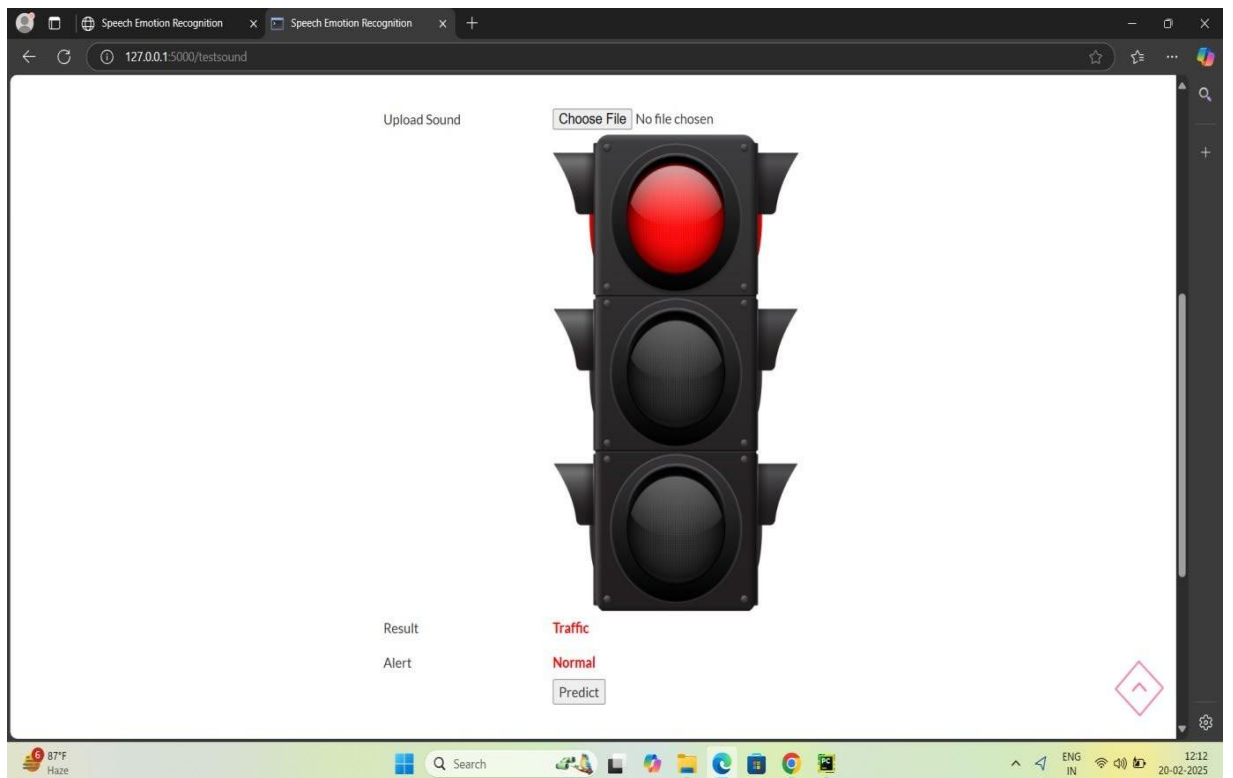


Fig. No. 10.14 Emergency Vehicle Not Detected

CHAPTER 11

REFERENCES

1. Aoki, Shunsuke, and Ragunathan Rajkumar. "Safe intersection management with cooperative perception for mixed traffic of human-driven and autonomous vehicles." *IEEE Open Journal of Vehicular Technology* 3 (2022): 251-265.
2. Arikumar, Kochupillai Selvaraj, et al. "V2x-based highly reliable warning system for emergency vehicles." *Applied Sciences* 13.3 (2023): 1950.
3. Butt, Faran Awais, et al. "On the integration of enabling wireless technologies and sensor fusion for next-generation connected and autonomous vehicles." *IEEE Access* 10 (2022): 14643-14668.
4. Chowdhury, Abdullahi, et al. "IoT-based emergency vehicle services in intelligent transportation system." *Sensors* 23.11 (2023): 5324.
5. Costa, Daniel G., et al. "A survey of emergencies management systems in smart cities." *IEEE Access* 10 (2022): 61843-61872.
6. Creß, Christian, Zhenshan Bing, and Alois C. Knoll. "Intelligent transportation systems using roadside infrastructure: A literature survey." *IEEE Transactions on Intelligent Transportation Systems* 25.7 (2023): 6309-6327.
7. Esenturk, Emre, et al. "Identification of traffic accident patterns via cluster analysis and test scenario development for autonomous vehicles." *IEEE Access* 10 (2022): 6660-6675.
8. Gholamhosseinian, Ashkan, and Jochen Seitz. "A[12] Esenturk, Emre, et al. "Identification of traffic accident patterns via cluster analysis and test scenario development for autonomous vehicles." *IEEE Access* 10 (2022): 6660-6675.

9. Jebamikyous, Hrag-Harout, and Rasha Kashef. "Autonomous vehicles perception (avp) using deep learning: Modeling, assessment, and challenges." *IEEE Access* 10 (2022): 10523-10535.
10. Jurczenia, Karol, and Jacek Rak. "A survey of vehicular network systems for road traffic management." *IEEE Access* 10 (2022): 42365- 42385.
11. Lin, Huei-Yung, and Cheng-Yu Ho. "Adaptive speed bump with vehicle identification for intelligent traffic flow control." *IEEE Access* 10 (2022): 68009-68016.
12. Mei, Hao, et al. "Libsignal: an open library for traffic signal control." *Machine Learning* 113.8 (2024): 5235-5271.
13. Mittal, Usha, and Priyanka Chawla. "Acoustic based emergency vehicle detection using ensemble of deep learning models." *Procedia Computer Science* 218 (2023): 227-234.
14. Naeem, Awad Bin, et al. "Intelligent road management system for autonomous, non-autonomous, and VIP vehicles." *World Electric Vehicle Journal* 14.9 (2023): 238.
15. Sim Kuan Goh, Hongqing Wang, Jun Kit Chaw, Ting Tin Tin, Liantao shi, Nannan Huang, Hong-Seng Gan: super resolution GAN and global aware object detection system for vehicle detection in complex traffic environments (2024) 344-248-4
16. Xia, Kun, et al. "IBGS: A wearable smart system to assist visually challenged." *IEEE Access* 10 (2022): 77810-77825.